

The University of Faisalabad

Name:

Humna Imran

Registration Number:

2023-BS-AI-017

Lab Manual:

Computer Vision

Course Code:

AI-311 Lab

Instructor:

Ms. Esha Nawaz

Department:

Computer Science

Degree:

BS Artificial Intelligence

Table of Content

Lab 1: Introduction to Computer Vision.....	3
Lab 2: Introduction to Deep Learning for Computer Vision	6
Lab 3: Corner Detection and Fourier Transform	11
Lab 4: Fourier transform.....	20
Lab 5: Corner and Edge Detection via Eigenvalue Analysis	26
Lab 6: Image Thresholding.....	32
Lab 7: Unsupervised Image Segmentation	37
Lab 8: Felzenszwalb Graph-Based Segmentation	43
Lab 9: Image Segmentation Pipeline	53
Lab 10: CIFAR-10 Image Classification using CNN	63
Lab 11: Image Classification using Pre-trained CNN (MobileNetV2).....	67
Lab 12: Simple Linear Regression with Scikit-Learn.....	72
Lab 13: Feature Extraction Using SIFT in OpenCV.....	74
Lab 15: Super-Fast Object Detection (YOLO)	81

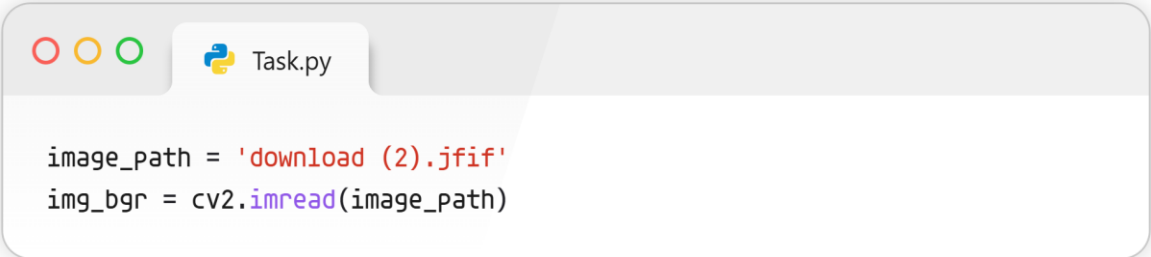
Lab 1: Introduction to Computer Vision

What We Are Doing & Why

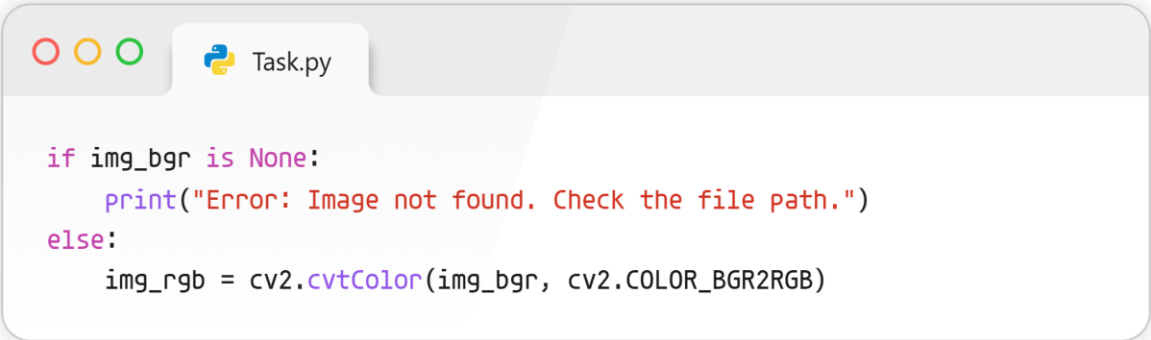
We are applying foundational image processing techniques—such as grayscale conversion, brightness adjustment, artificial occlusion, and edge detection—to a sample image. We are doing this to demonstrate how different computer vision filters manipulate image data to extract structural features or simulate varying visual conditions, like lighting changes and blocked views. In summary, this script serves as a practical demonstration that systematically transforms an image using OpenCV and visualizes the comparative results side-by-side, providing a clear understanding of how these core operations affect raw pixel data.



```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```



```
image_path = 'download (2).jfif'
img_bgr = cv2.imread(image_path)
```



```
if img_bgr is None:
    print("Error: Image not found. Check the file path.")
else:
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```



Task.py

```
gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)
bright_img = cv2.convertScaleAbs(img_rgb, alpha=1.2, beta=30)
occluded_img = img_rgb.copy()
cv2.rectangle(occluded_img, (100, 100), (300, 300), (0, 0, 0), -1)
edges = cv2.Canny(gray, 100, 200)
```



Task.py

```
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("1. Original RGB")
plt.axis('off')

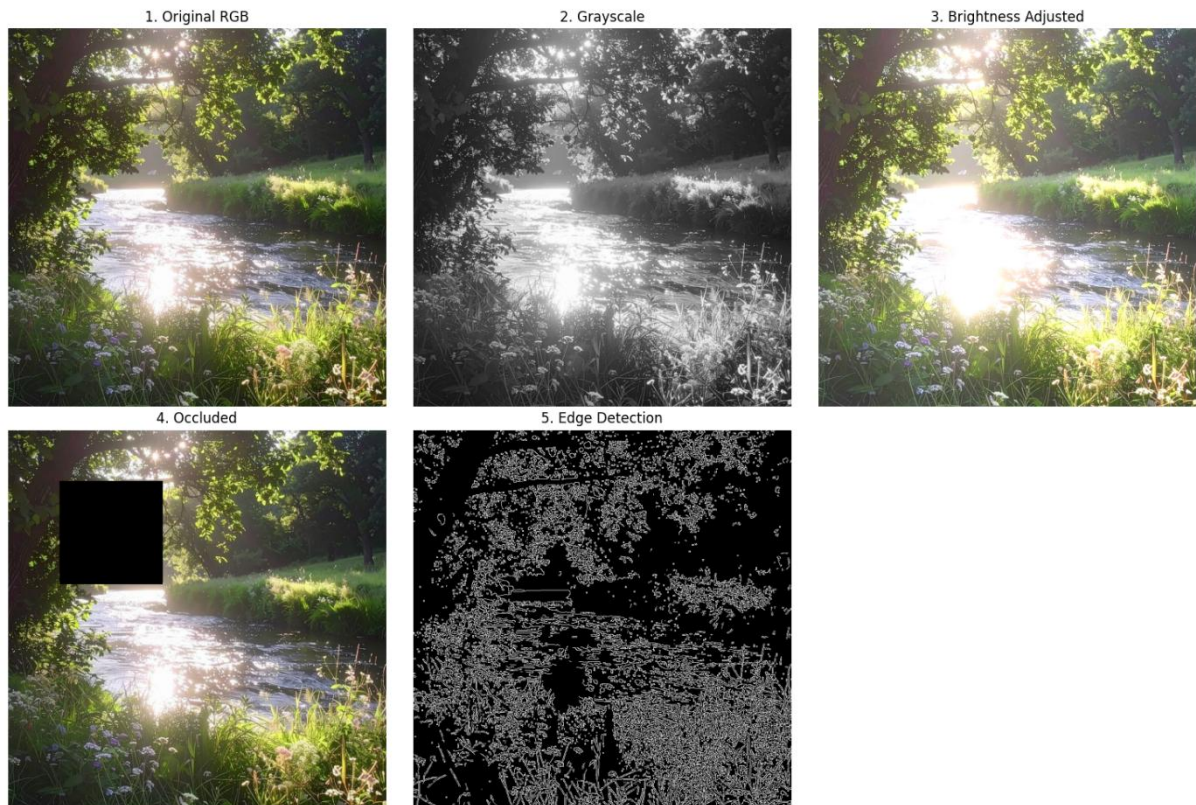
plt.subplot(2, 3, 2)
plt.imshow(gray, cmap='gray')
plt.title("2. Grayscale")
plt.axis('off')

plt.subplot(2, 3, 3)
plt.imshow(bright_img)
plt.title("3. Brightness Adjusted")
plt.axis('off')

plt.subplot(2, 3, 4)
plt.imshow(occluded_img)
plt.title("4. Occluded")
plt.axis('off')

plt.subplot(2, 3, 5)
plt.imshow(edges, cmap='gray')
plt.title("5. Edge Detection")
plt.axis('off')

plt.tight_layout()
plt.show()
```

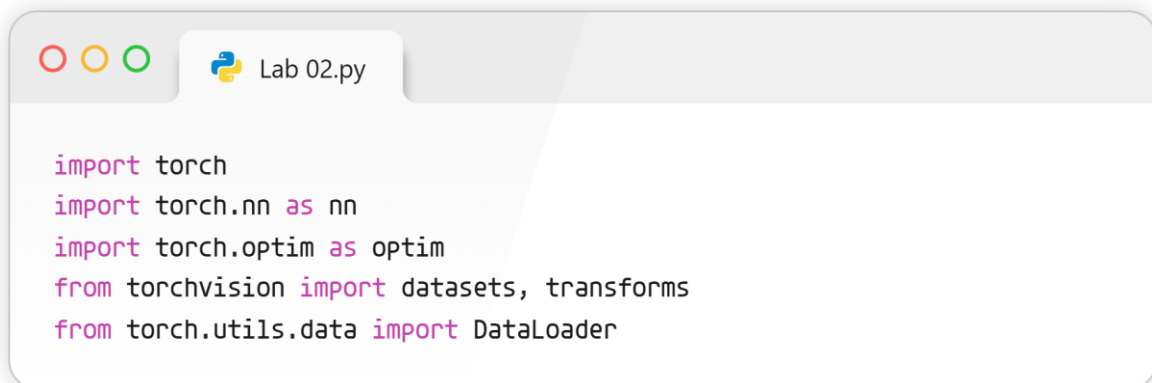
Summary

This code represents a fundamental image processing pipeline using Python, OpenCV, and Matplotlib. It demonstrates how to read an image, manipulate its color spaces, alter its pixel values to simulate real-world conditions (brightness and occlusion), extract structural features (edges), and finally visualize all these different states side-by-side for comparison.

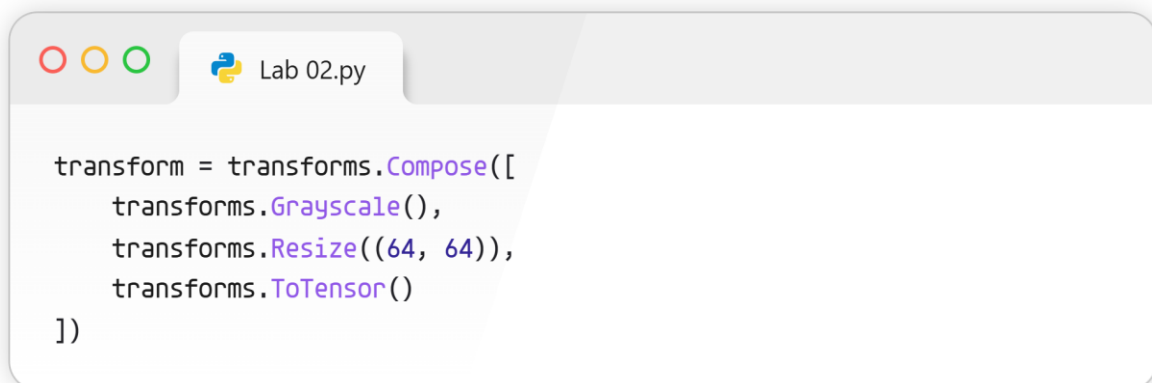
Lab 2: Introduction to Deep Learning for Computer Vision

What are we doing and Why?

We are building an end-to-end computer vision pipeline that bridges traditional image processing with modern deep learning. First, we use OpenCV to apply foundational techniques, such as grayscale conversion, brightness adjustment, artificial occlusion, and edge detection, to understand how raw pixel data can be manipulated and how structural features are extracted. Then, we transition into machine learning by constructing, training, and evaluating two different neural network architectures (a Multi-Layer Perceptron and a Convolutional Neural Network) using PyTorch. We are doing this to demonstrate the complete workflow of a computer vision project, from preparing and visualizing raw visual data to teaching AI models how to automatically classify those images, ultimately allowing us to compare how different network designs handle visual tasks.



```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```



```
transform = transforms.Compose([
    transforms.Grayscale(),
    transforms.Resize((64, 64)),
    transforms.ToTensor()
])
```

Lab 02.py

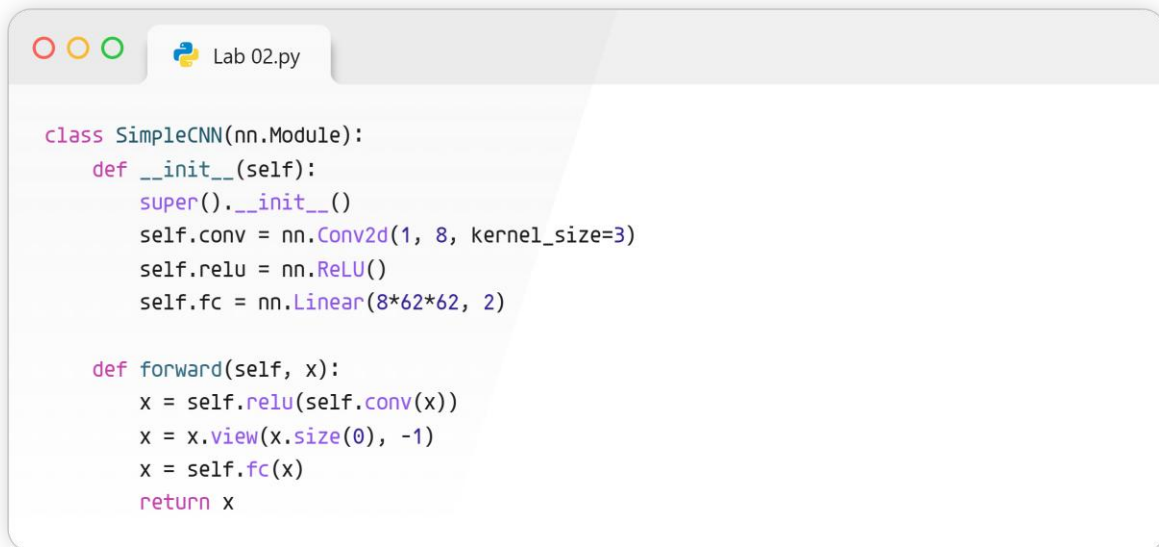
```
train_data = datasets.CIFAR10(  
    root="./data",  
    train=True,  
    download=True,  
    transform=transform  
)  
  
test_data = datasets.CIFAR10(  
    root="./data",  
    train=False,  
    download=True,  
    transform=transform  
)
```

Lab 02.py

```
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)  
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)  
  
print(f"Train dataset size: {len(train_data)}")  
print(f"Test dataset size: {len(test_data)}")
```

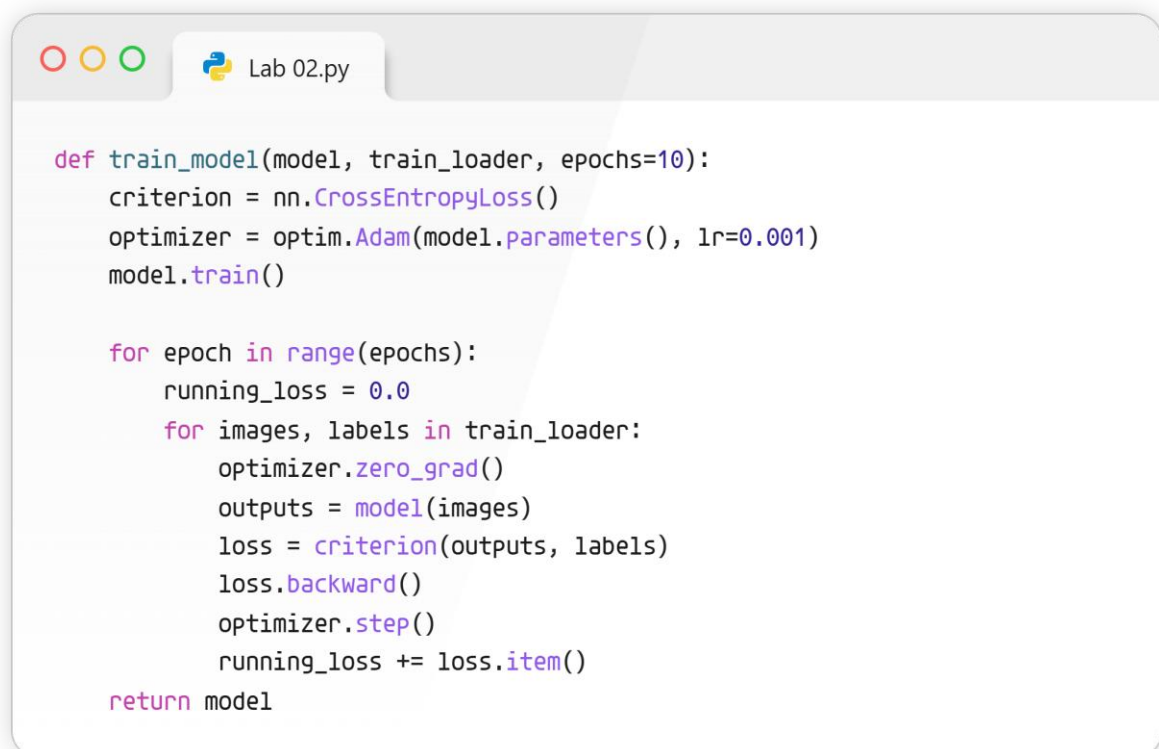
Lab 02.py

```
class SimpleMLP(nn.Module):  
    def __init__(self):  
        super().__init__()  
  
        self.flatten = nn.Flatten()  
        self.fc1 = nn.Linear(4096, 128)  
        self.relu = nn.ReLU()  
        self.fc2 = nn.Linear(128, 2)  
  
    def forward(self, x):  
        x = self.flatten(x)  
        x = self.fc1(x)  
        x = self.relu(x)  
        x = self.fc2(x)  
        return x
```



```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 8, kernel_size=3)
        self.relu = nn.ReLU()
        self.fc = nn.Linear(8*62*62, 2)

    def forward(self, x):
        x = self.relu(self.conv(x))
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x
```



```
def train_model(model, train_loader, epochs=10):
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    model.train()

    for epoch in range(epochs):
        running_loss = 0.0
        for images, labels in train_loader:
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
        return model
```

```
Lab 02.py

def evaluate_model(model, test_loader):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in test_loader:
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    return 100 * correct / total
```

```
Lab 02.py

train_loader = DataLoader(train_data, batch_size=8, shuffle=True)
test_loader = DataLoader(test_data, batch_size=8)

cnn_model = SimpleCNN()
mlp_model = SimpleMLP()

print("Training CNN...")
train_model(cnn_model, train_loader)
cnn_acc = evaluate_model(cnn_model, test_loader)

print("Training Simple MLP...")
train_model(mlp_model, train_loader)
mlp_acc = evaluate_model(mlp_model, test_loader)
```

```
Lab 02.py

print("\n--- Final Performance Comparison ---")
print(f"CNN Accuracy: {cnn_acc:.2f}%")
print(f"MLP Accuracy: {mlp_acc:.2f}%")
```

```
100%|██████████| 170M/170M [00:05<00:00, 28.7MB/s]
```

```
Train dataset size: 50000
```

```
Test dataset size: 10000
```

```
DEBUG (DataLoader): Max label in train_loader: 9
```

```
DEBUG (Model Init): CNN model final layer output features: 10
```

```
DEBUG (Model Init): MLP model final layer output features: 10
```

```
Training CNN...
```

```
DEBUG_TRAIN (Before Model): images.shape: torch.Size([8, 1, 64, 64])
```

```
DEBUG_TRAIN (After Model): outputs.shape: torch.Size([8, 10])
```

```
DEBUG_TRAIN (Labels): labels.shape: torch.Size([8])
```

```
DEBUG_TRAIN (Labels): labels.min(): 2
```

```
DEBUG_TRAIN (Labels): labels.max(): 9
```

```
DEBUG_TRAIN (Labels): labels: tensor([6, 9, 4, 5, 3, 9, 3, 2])
```

```
Training Simple MLP...
```

```
DEBUG_TRAIN (Before Model): images.shape: torch.Size([8, 1, 64, 64])
```

```
DEBUG_TRAIN (After Model): outputs.shape: torch.Size([8, 10])
```

```
DEBUG_TRAIN (Labels): labels.shape: torch.Size([8])
```

```
DEBUG_TRAIN (Labels): labels.min(): 0
```

```
DEBUG_TRAIN (Labels): labels.max(): 9
```

```
DEBUG_TRAIN (Labels): labels: tensor([3, 4, 4, 9, 0, 5, 7, 7])
```

```
--- Final Performance Comparison ---
```

```
CNN Accuracy: 10.22%
```

```
MLP Accuracy: 10.05%
```

Summary

In summary, this project explores both foundational and advanced computer vision techniques in a single workflow. We begin by applying standard OpenCV filters to manipulate image data and visualize the effects of lighting, occlusion, and edge detection. Building on this, we set up a PyTorch environment to train and evaluate a standard MLP and a CNN on an image classification dataset. By comparing these models side-by-side, we gain practical insight into how deep learning architectures process visual features and why convolutional layers are specifically suited for image recognition.

LAB 3: Corner Detection and Fourier Transform

What We Are Doing and Why

Did you know images have "frequencies"? In an image, **Low Frequency** means smooth areas (like a clear blue sky). **High Frequency** means sharp changes and details (like the individual hairs on a dog). We will use a math spell called the **Fourier Transform** to split the image into these frequencies, and then apply a filter (like sunglasses for the computer) to blur the image!

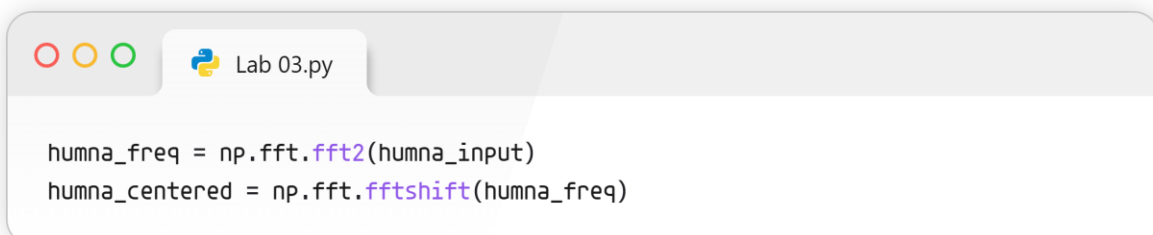
The Code

A code editor window titled "Lab 03.py" with a Python icon. It contains the following Python code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

A code editor window titled "Lab 03.py" with a Python icon. It contains the following Python code:

```
humna_input = cv2.imread('image.jpg', 0)
```

A code editor window titled "Lab 03.py" with a Python icon. It contains the following Python code:

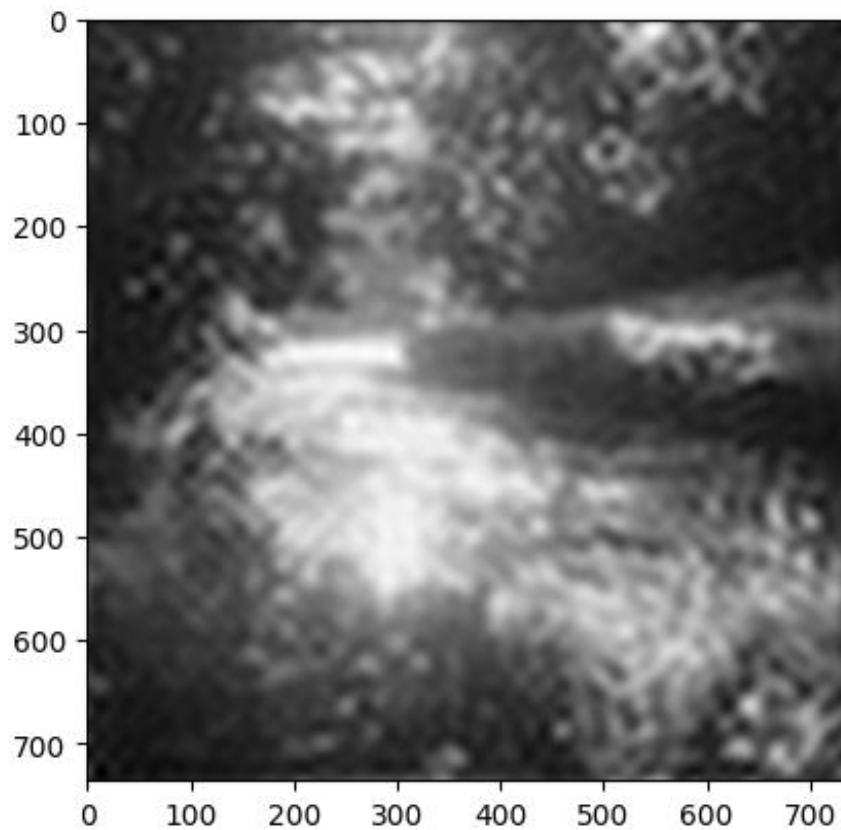
```
humna_freq = np.fft.fft2(humna_input)
humna_centered = np.fft.fftshift(humna_freq)
```

```
rows, cols = humna_input.shape
crow, ccol = rows//2, cols//2
humna_mask = np.zeros((rows, cols), np.uint8)
humna_mask[crow-30:crow+30, ccol-30:ccol+30] = 1
```

```
humna_filtered = humna_centered * humna_mask
```

```
humna_shifted_back = np.fft.ifftshift(humna_filtered)
humna_final_image = np.fft.ifft2(humna_shifted_back)
humna_final_image = np.abs(humna_final_image)
```

```
plt.imshow(humna_final_image, cmap='gray')
plt.show()
```

```
Lab 03.py

img = cv2.imread('BUIDINGG.jfif', 0)
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)

magnitude_spectrum = 20 * np.log(np.abs(fshift))
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2
radius = 30

lpf_mask = np.zeros((rows, cols), np.uint8)
lpf_mask[crow-radius : crow+radius, ccol-radius : ccol+radius] = 1
lpf_fshift = fshift * lpf_mask

hpf_mask = np.ones((rows, cols), np.uint8)
hpf_mask[crow-radius : crow+radius, ccol-radius : ccol+radius] = 0
hpf_fshift = fshift * hpf_mask
```

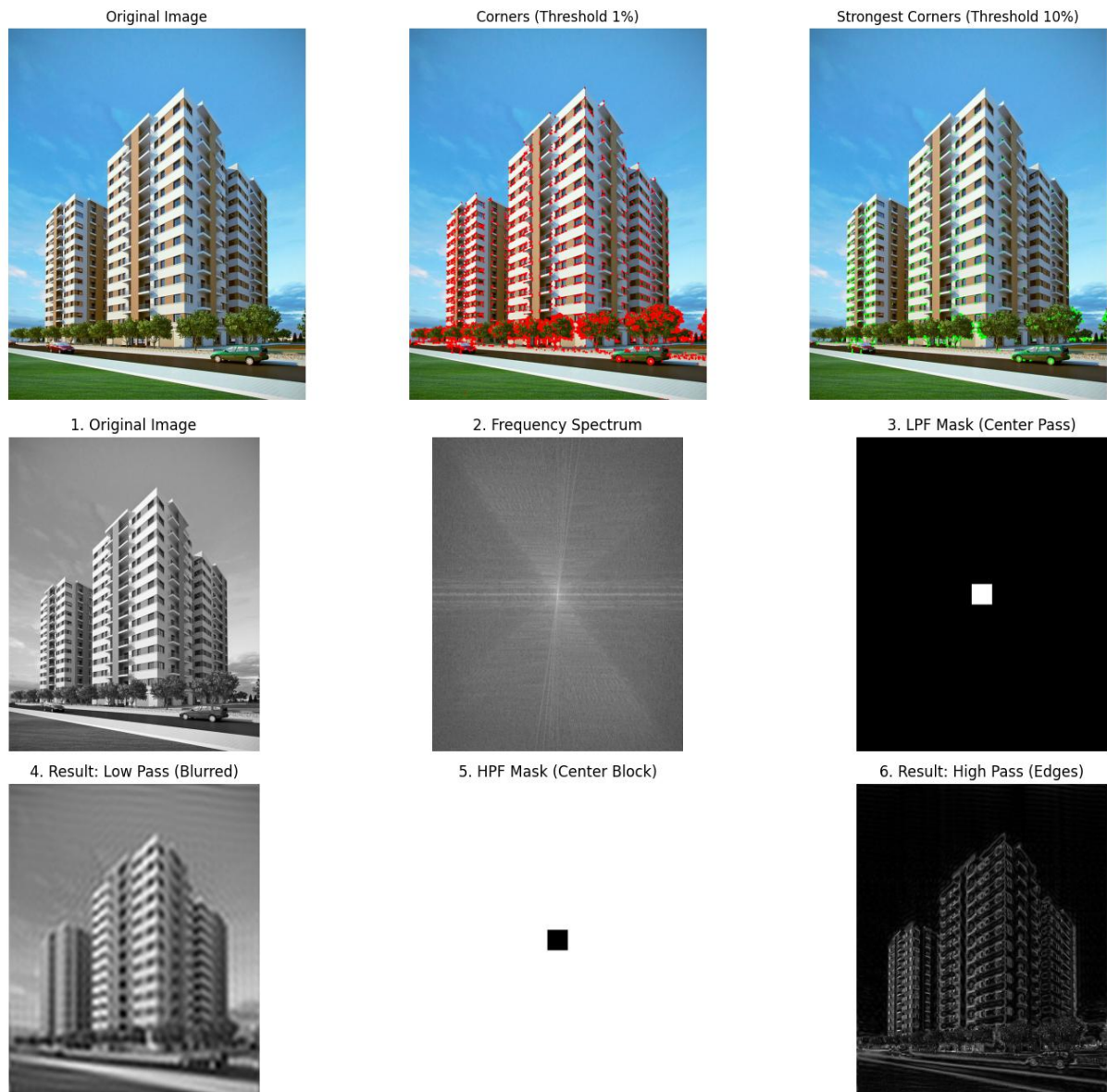


Lab 03.py

```
def inverse_fft(shifted_data):
    f_ishift = np.fft.ifftshift(shifted_data)
    img_back = np.fft.ifft2(f_ishift)
    return np.abs(img_back)

img_lpf = inverse_fft(lpf_fshift)
img_hpf = inverse_fft(hpf_fshift)
plt.figure(figsize=(16, 8))
plt.subplot(2, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("1. Original Image")
plt.axis('off')
plt.subplot(2, 3, 2)
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title("2. Frequency Spectrum")
plt.axis('off')
plt.subplot(2, 3, 3)
plt.imshow(lpf_mask, cmap='gray')
plt.title("3. LPF Mask (Center Pass)")
plt.axis('off')

plt.subplot(2, 3, 4)
plt.imshow(img_lpf, cmap='gray')
plt.title("4. Result: Low Pass (Blurred)")
plt.axis('off')
plt.subplot(2, 3, 5)
plt.imshow(hpf_mask, cmap='gray')
plt.title("5. HPF Mask (Center Block)")
plt.axis('off')
plt.subplot(2, 3, 6)
plt.imshow(img_hpf, cmap='gray')
plt.title("6. Result: High Pass (Edges)")
plt.axis('off')
plt.tight_layout()
plt.show()
```



Summary

We converted the picture into waves and frequencies. Then, we made a digital "mask" that blocked all the high frequencies (the sharp edges and details). When we turned the frequencies back into a normal picture, the image became super blurry! This is called a **Low Pass Filter**.



Lab 03.py

```
dst = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
dst = cv2.dilate(dst, None)

marked_img = original_rgb.copy()
marked_img[dst > 0.01 * dst.max()] = [255, 0, 0]
strong_corners = original_rgb.copy()
strong_corners[dst > 0.1 * dst.max()] = [0, 255, 0]
plt.figure(figsize=(15, 5))

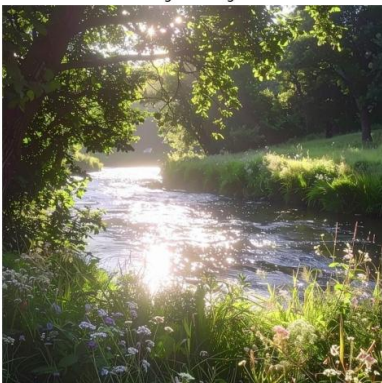
plt.subplot(1, 3, 1)
plt.imshow(original_rgb)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(marked_img)
plt.title("Corners (Threshold 1%)")
plt.axis('off')

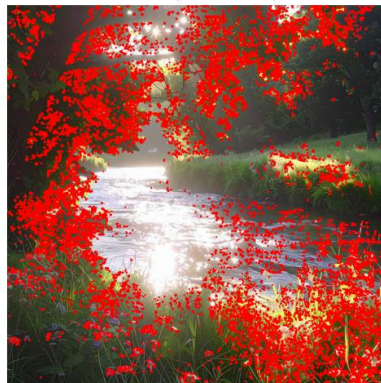
plt.subplot(1, 3, 3)
plt.imshow(strong_corners)
plt.title("Strongest Corners (Threshold 10%)")
plt.axis('off')

plt.tight_layout()
plt.show()
```

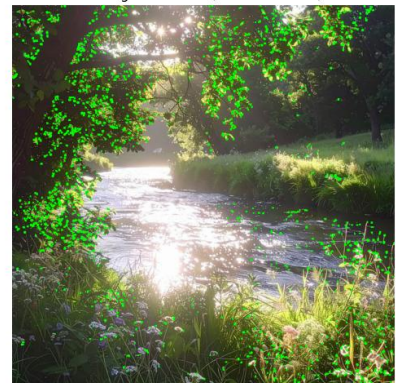
Original Image

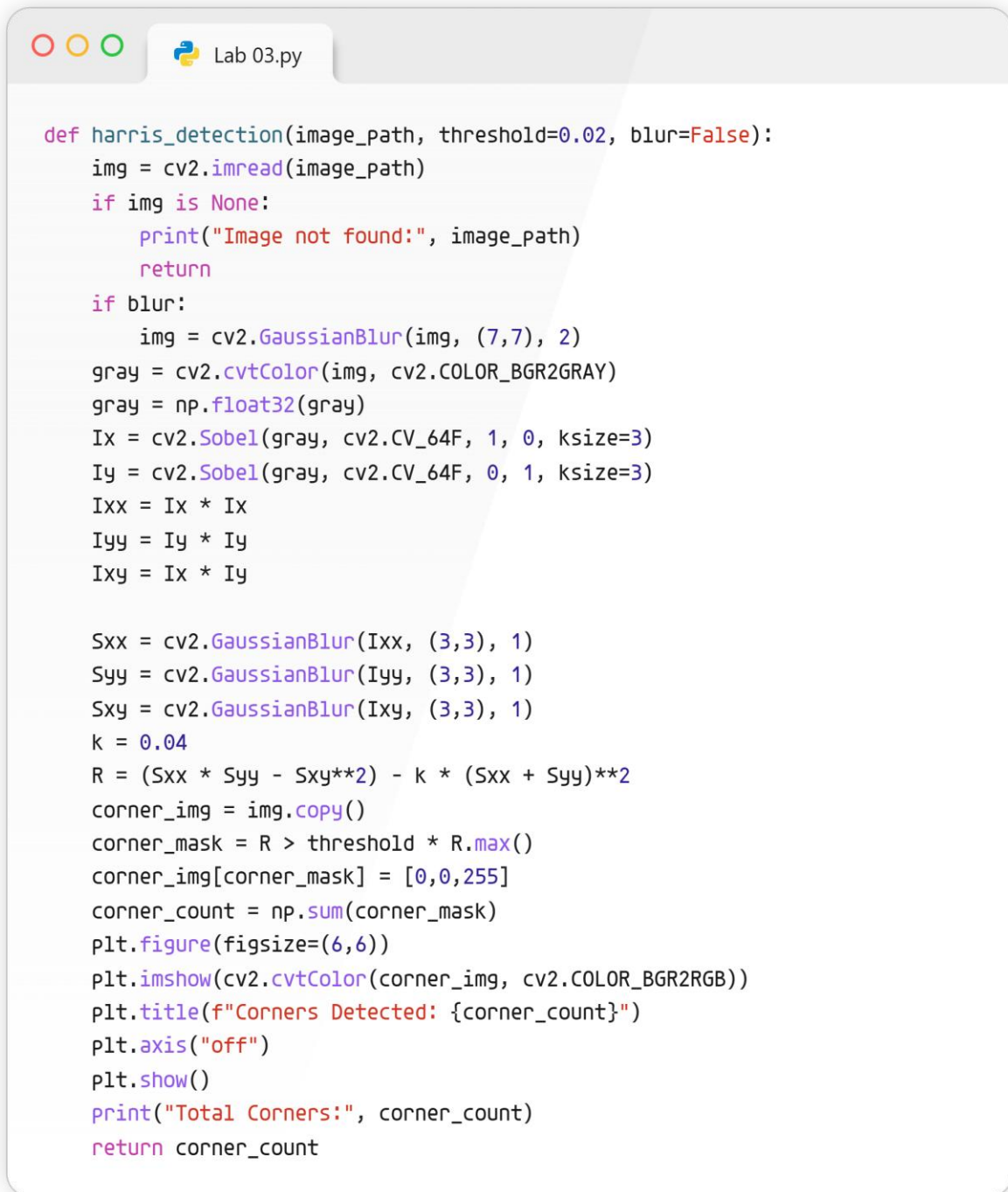


Corners (Threshold 1%)



Strongest Corners (Threshold 10%)





```
def harris_detection(image_path, threshold=0.02, blur=False):
    img = cv2.imread(image_path)
    if img is None:
        print("Image not found:", image_path)
        return
    if blur:
        img = cv2.GaussianBlur(img, (7,7), 2)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = np.float32(gray)
    Ix = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    Iy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
    Ixx = Ix * Ix
    Iyy = Iy * Iy
    Ixy = Ix * Iy

    Sxx = cv2.GaussianBlur(Ixx, (3,3), 1)
    Syy = cv2.GaussianBlur(Iyy, (3,3), 1)
    Sxy = cv2.GaussianBlur(Ixy, (3,3), 1)
    k = 0.04
    R = (Sxx * Syy - Sxy**2) - k * (Sxx + Syy)**2
    corner_img = img.copy()
    corner_mask = R > threshold * R.max()
    corner_img[corner_mask] = [0,0,255]
    corner_count = np.sum(corner_mask)
    plt.figure(figsize=(6,6))
    plt.imshow(cv2.cvtColor(corner_img, cv2.COLOR_BGR2RGB))
    plt.title(f"Corners Detected: {corner_count}")
    plt.axis("off")
    plt.show()
    print("Total Corners:", corner_count)
    return corner_count
```



```
print("Building Image")
building_corners = harris_detection("BUILDING.jfif", threshold=0.02)
print("Road Image")
road_corners = harris_detection("ROAD IMG.jfif", threshold=0.02)
```


Corners Detected: 13345



Corners Detected: 19481



Summary

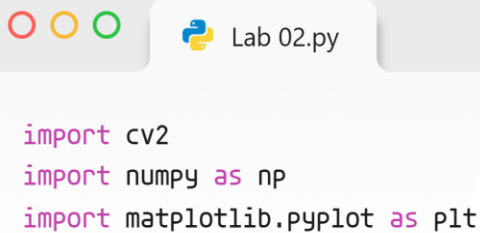
This code mathematically checks every single pixel in the image. If a pixel looks like a flat wall, it ignores it. If it looks like a straight edge, it ignores it. But if it finds a sharp, pointy corner, it turns that pixel bright red so we can see it!

LAB 4: Fourier transform

What We Are Doing and Why

Edges are cool, but **Corners** are even better! A corner is a spot where lines meet (like the pointy tip of a square). Computers love corners because they don't change even if you move the camera. We will use the **Harris Corner Detector**, which finds points where the color changes in *two* directions at once.

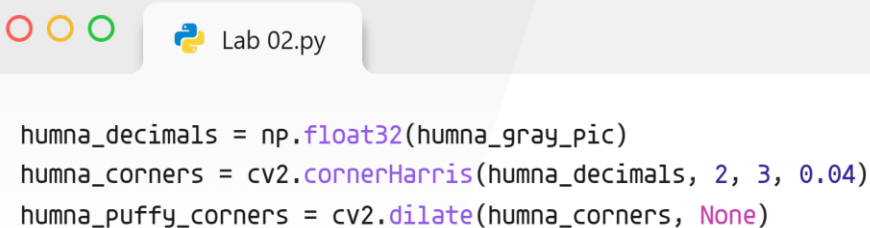
The Code



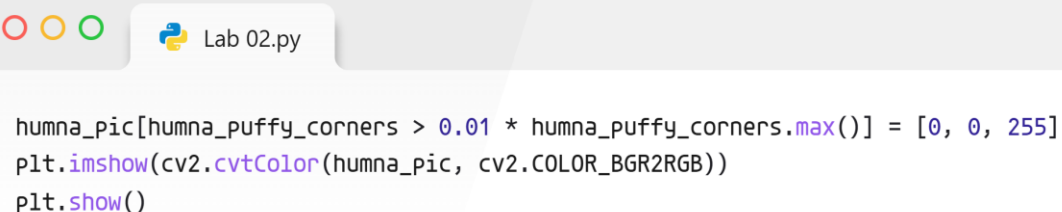
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```



```
humna_pic = cv2.imread('building.jpg')
humna_gray_pic = cv2.cvtColor(humna_pic, cv2.COLOR_BGR2GRAY)
```



```
humna_decimals = np.float32(humna_gray_pic)
humna_corners = cv2.cornerHarris(humna_decimals, 2, 3, 0.04)
humna_puffy_corners = cv2.dilate(humna_corners, None)
```



```
humna_pic[humna_puffy_corners > 0.01 * humna_puffy_corners.max()] = [0, 0, 255]
plt.imshow(cv2.cvtColor(humna_pic, cv2.COLOR_BGR2RGB))
plt.show()
```






```
Ix = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
Iy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

Ixx = Ix * Ix
Iyy = Iy * Iy
Ixy = Ix * Iy

Sxx = cv2.GaussianBlur(Ixx, (3,3), 1)
Syy = cv2.GaussianBlur(Iyy, (3,3), 1)
Sxy = cv2.GaussianBlur(Ixy, (3,3), 1)
height, width = gray.shape

corner_img = img.copy()
edge_img = img.copy()
flat_img = img.copy()

threshold = 0.01 * Sxx.max()

for y in range(height):
    for x in range(width):

        M = np.array([[Sxx[y,x], Sxy[y,x]],
                      [Sxy[y,x], Syy[y,x]]])

        eigenvalues, _ = np.linalg.eig(M)
        lambda1, lambda2 = eigenvalues

        if lambda1 > threshold and lambda2 > threshold:
            corner_img[y,x] = [0,0,255]    # Red (corner)

        elif (lambda1 > threshold and lambda2 <= threshold) or \
             (lambda2 > threshold and lambda1 <= threshold):
            edge_img[y,x] = [255,0,0]      # Blue (edge)

        else:
            flat_img[y,x] = [0,255,0]      # Green (flat)
```

```
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.title("Corners (Red)")
plt.imshow(cv2.cvtColor(corner_img, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.subplot(1,3,2)
plt.title("Edges (Blue)")
plt.imshow(cv2.cvtColor(edge_img, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.subplot(1,3,3)
plt.title("Flat Regions (Green)")
plt.imshow(cv2.cvtColor(flat_img, cv2.COLOR_BGR2RGB))
plt.axis("off")

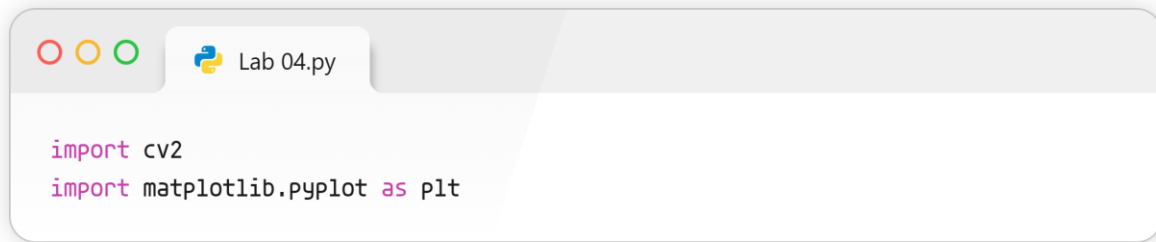
plt.show()
```



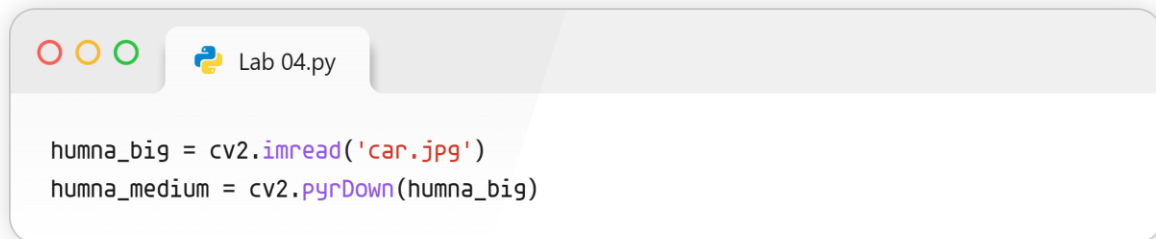
What We Are Doing and Why

Sometimes objects in a picture are way too big or way too small. An **Image Pyramid** is a way to make a bunch of copies of a picture, where each copy is half the size of the last one. It looks like a pyramid if you stack them! This helps computers look for objects at different sizes.

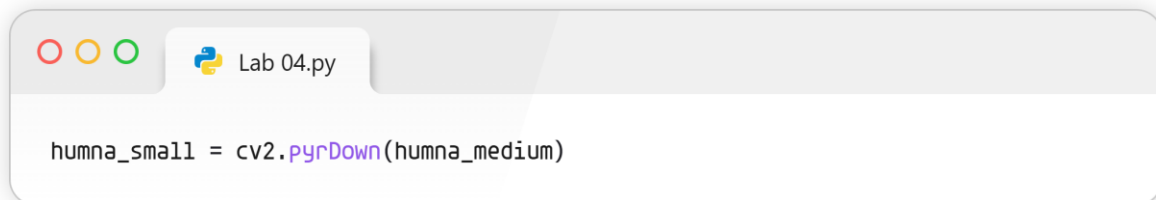
The Code



```
import cv2
import matplotlib.pyplot as plt
```



```
humna_big = cv2.imread('car.jpg')
humna_medium = cv2.pyrDown(humna_big)
```

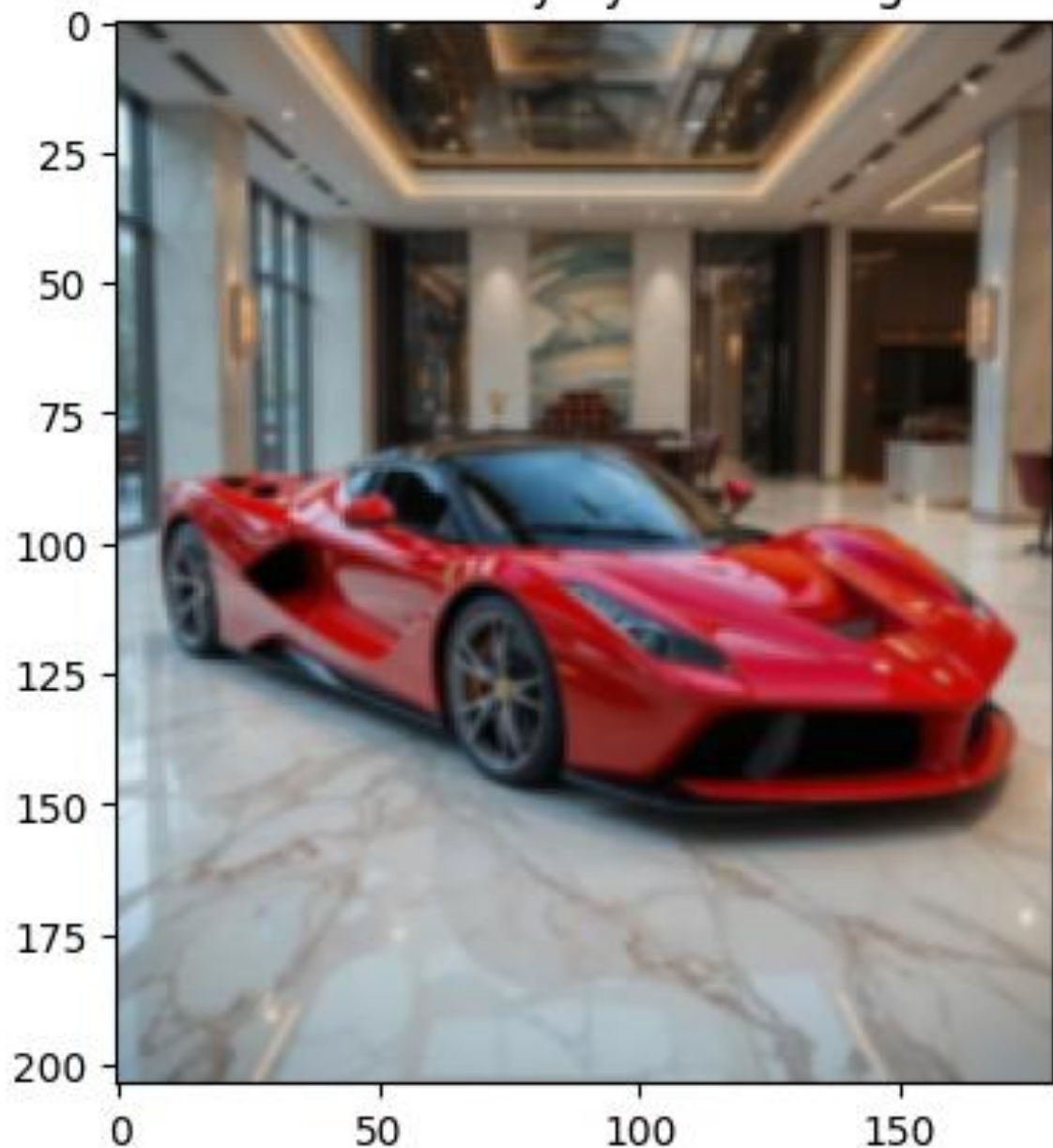


```
humna_small = cv2.pyrDown(humna_medium)
```



```
plt.imshow(cv2.cvtColor(humna_small, cv2.COLOR_BGR2RGB))
plt.title("Humna's Tiny Pyramid Image")
plt.show()
```

Humna's Tiny Pyramid Image



Summary

By using `pyrDown`, we created a Gaussian Pyramid. The computer slightly blurred the image to prevent glitchy pixels, and then shrunk it in half. We did this twice to get a very small, smooth picture.

LAB 5: Corner and Edge Detection via Eigenvalue Analysis

What We Are Doing and Why

We are going to open a picture using Python and look at its true form: numbers! Computers see colors as a mix of Red, Green, and Blue (RGB). We will also change the picture into Grayscale (black and white) and find the edges (outlines) of objects. Finding edges is important because it helps the computer ignore the colors and focus on the shapes.

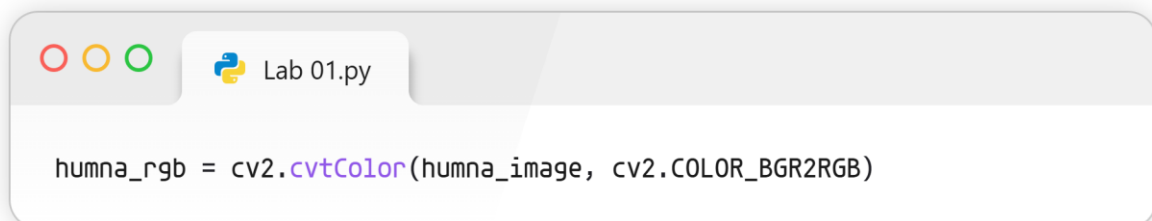
The Code



```
import cv2
import matplotlib.pyplot as plt
```



```
humna_image = cv2.imread('image.jpg')
```

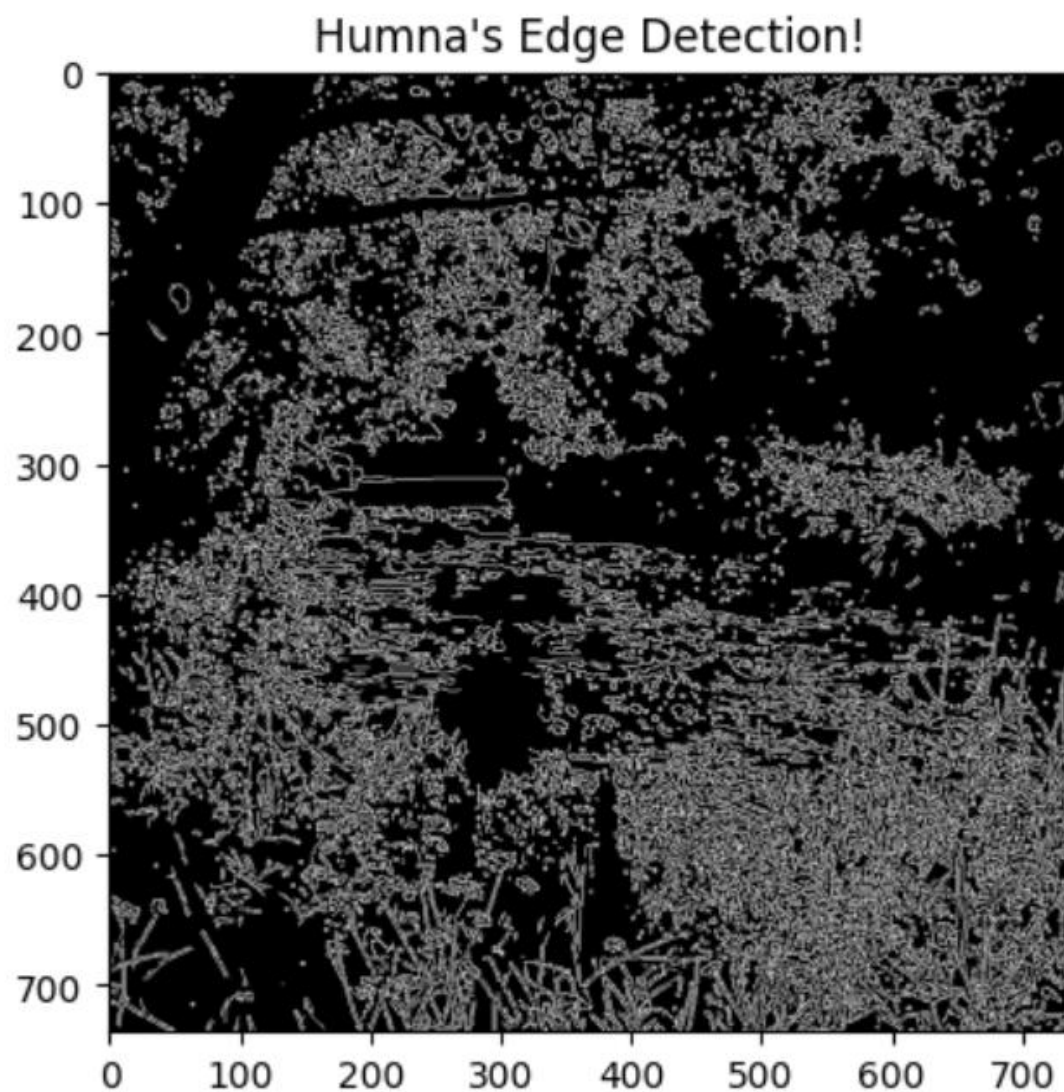


```
humna_rgb = cv2.cvtColor(humna_image, cv2.COLOR_BGR2RGB)
```



```
print(humna_rgb.shape)
humna_gray = cv2.cvtColor(humna_rgb, cv2.COLOR_RGB2GRAY)
humna_edges = cv2.Canny(humna_gray, 100, 200)
plt.imshow(humna_edges, cmap='gray')
plt.title("Humna's Edge Detection!")
plt.show()
```

(736, 736, 3)




```

import cv2
import numpy as np
import os
import urllib.request
import matplotlib.pyplot as plt
import urllib.request
import os
os.makedirs("models", exist_ok=True)
urls = {
    "MobileNetSSD_deploy.prototxt": "https://raw.githubusercontent.com/chuanqi305/MobileNet-SSD/master/voc/MobileNetSSD_deploy.prototxt",
    "MobileNetSSD_deploy.caffemodel": "https://github.com/chuanqi305/MobileNet-SSD/raw/master/voc/MobileNetSSD_deploy.caffemodel"
}

for name, url in urls.items():
    print(f"Downloading {name}...")
    urllib.request.urlretrieve(url, f"models/{name}")

print("Download complete. You can now run the detection script.")

```

```

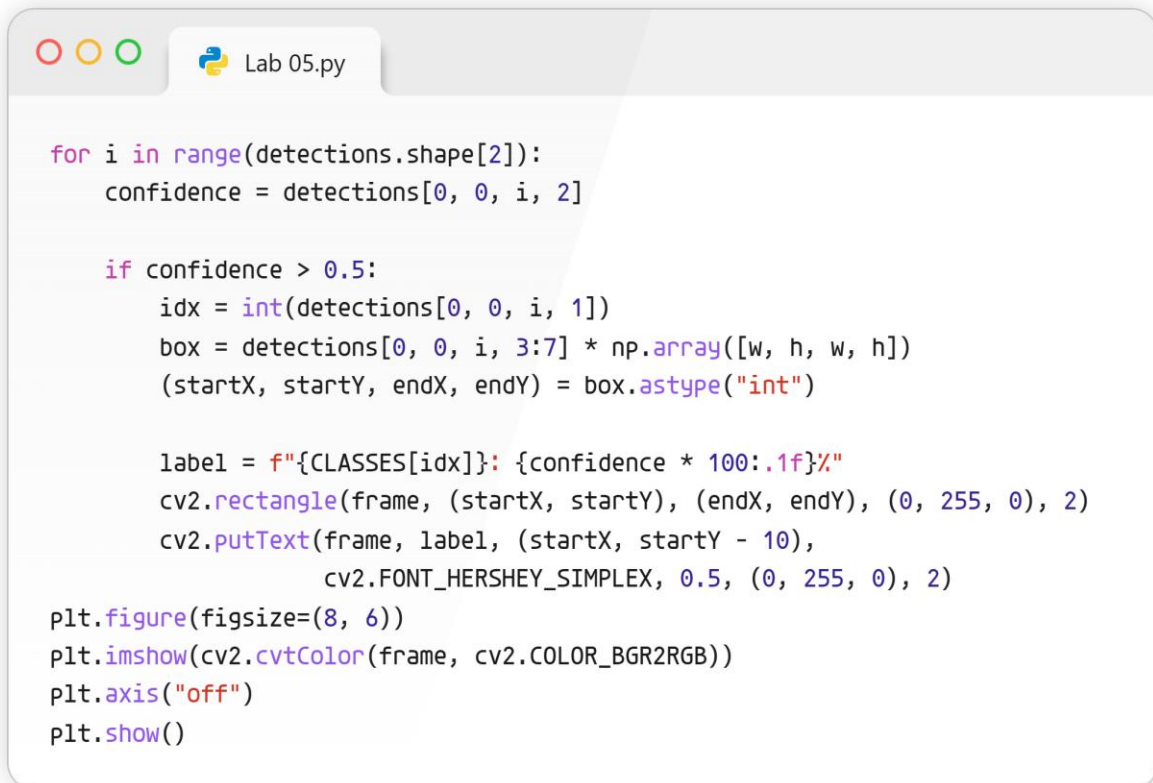
prototxt = "models/MobileNetSSD_deploy.prototxt"
caffemodel = "models/MobileNetSSD_deploy.caffemodel"

if not os.path.exists(prototxt) or not os.path.exists(caffemodel):
    print("Model files missing!")
else:
    net = cv2.dnn.readNetFromCaffe(prototxt, caffemodel)
    print("Model Loaded Successfully")
CLASSES = ["background", "aeroplane", "bicycle", "bird", "boat",
            "bottle", "bus", "car", "cat", "chair", "cow", "diningtable",
            "dog", "horse", "motorbike", "person", "pottedplant",
            "sheep", "sofa", "train", "tvmonitor"]
frame = cv2.imread("imagee.jfif")

if frame is None:
    print("Image not found!")
else:
    (h, w) = frame.shape[:2]
    blob = cv2.dnn.blobFromImage(
        cv2.resize(frame, (300, 300)),
        0.007843,
        (300, 300),
        127.5
    )

    net.setInput(blob)
    detections = net.forward()

```



```
for i in range(detections.shape[2]):
    confidence = detections[0, 0, i, 2]

    if confidence > 0.5:
        idx = int(detections[0, 0, i, 1])
        box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
        (startX, startY, endX, endY) = box.astype("int")

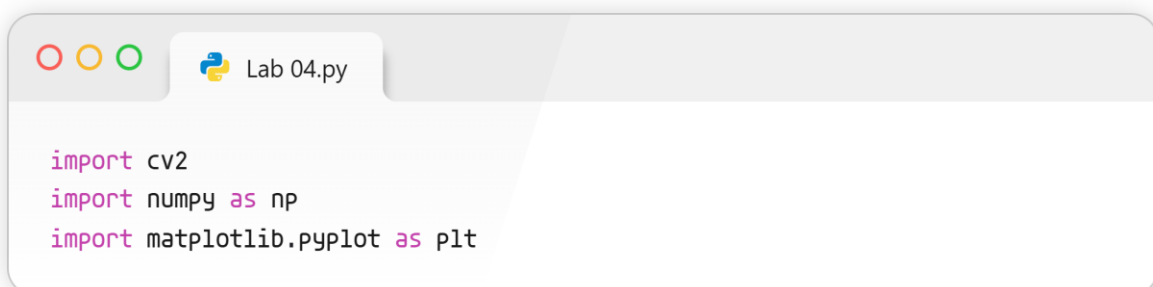
        label = f"{CLASSES[idx]}: {confidence * 100:.1f}%"
        cv2.rectangle(frame, (startX, startY), (endX, endY), (0, 255, 0), 2)
        cv2.putText(frame, label, (startX, startY - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

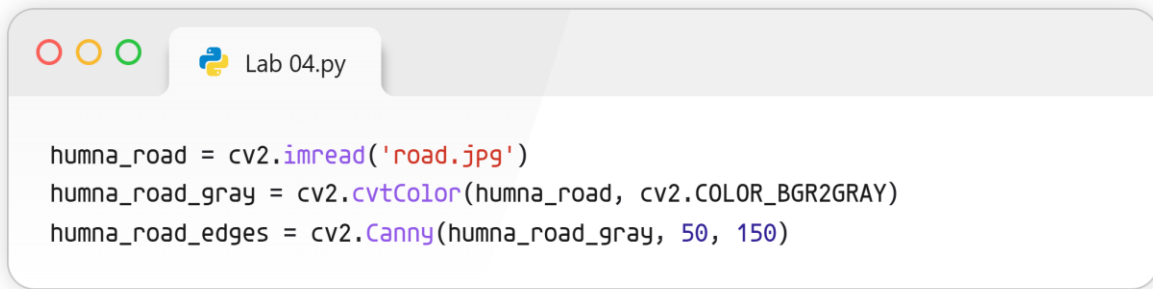
What We Are Doing and Why

If a computer wants to play a driving game, it needs to find the straight lines on the road. We use the **Hough Transform** for this. First, we find the edges, and then the computer makes all the edge pixels "vote" on whether they form a straight line together.

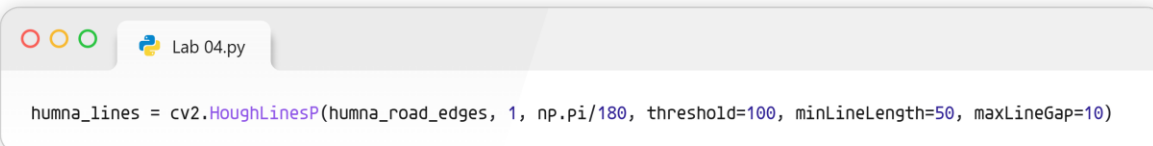
The Code



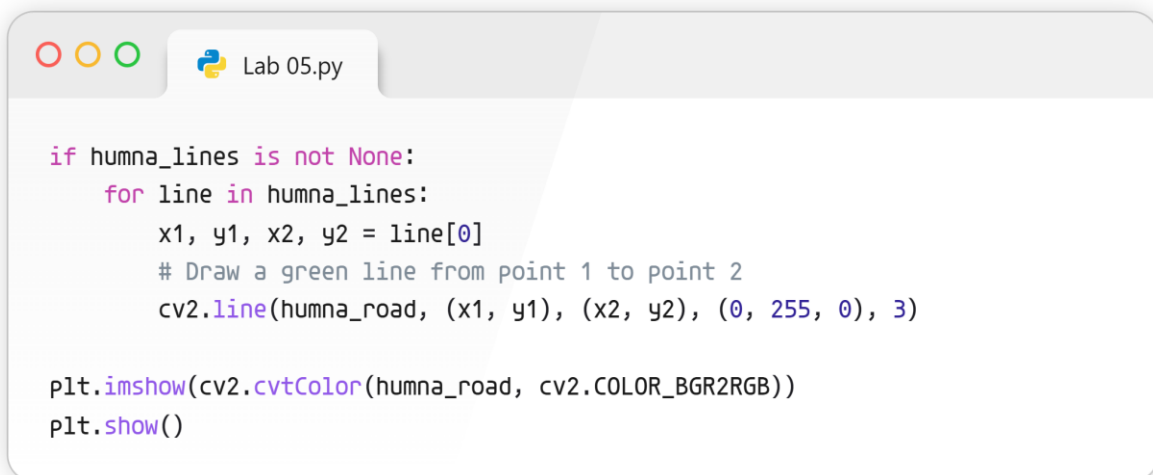
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```



```
humna_road = cv2.imread('road.jpg')
humna_road_gray = cv2.cvtColor(humna_road, cv2.COLOR_BGR2GRAY)
humna_road_edges = cv2.Canny(humna_road_gray, 50, 150)
```

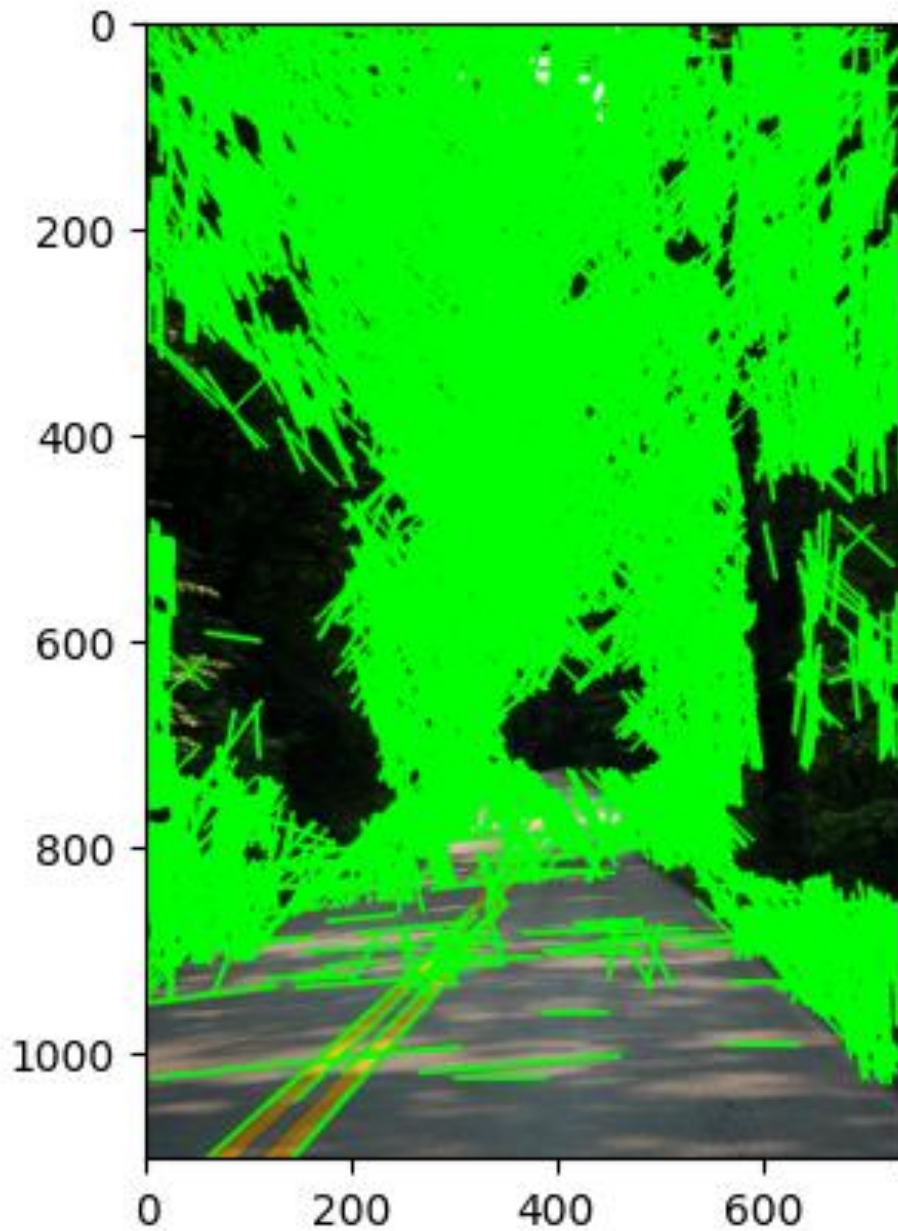


```
humna_lines = cv2.HoughLinesP(humna_road_edges, 1, np.pi/180, threshold=100, minLineLength=50, maxLineGap=10)
```



```
if humna_lines is not None:
    for line in humna_lines:
        x1, y1, x2, y2 = line[0]
        # Draw a green line from point 1 to point 2
        cv2.line(humna_road, (x1, y1), (x2, y2), (0, 255, 0), 3)

plt.imshow(cv2.cvtColor(humna_road, cv2.COLOR_BGR2RGB))
plt.show()
```



Summary

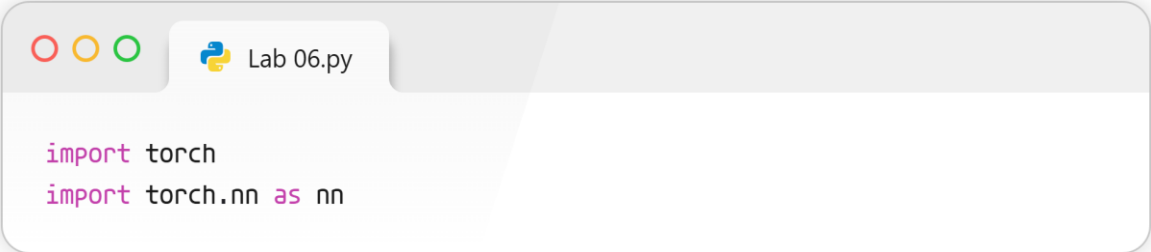
We found the outlines of the road using Canny Edge Detection. Then, the Hough Transform connected the dots. If enough dots formed a straight path, the computer recognized it as a line and drew a bright green line over it.

LAB 6: Image Thresholding

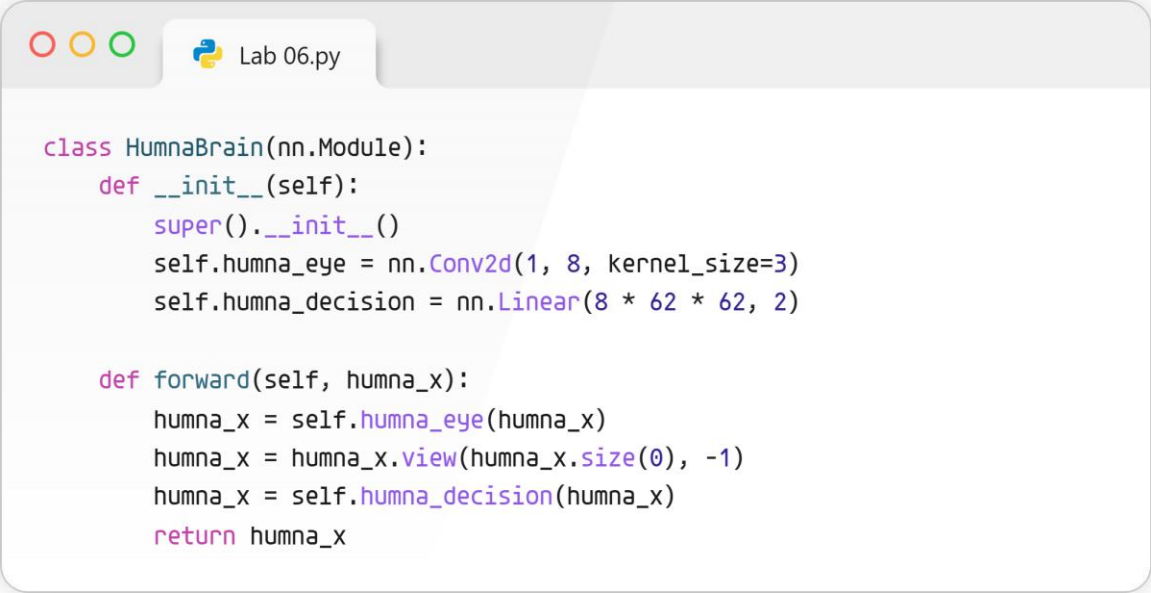
What We Are Doing and Why

In the past, humans had to tell computers *exactly* what to look for. Today, we use **Deep Learning**. We build tiny artificial brains called **Convolutional Neural Networks (CNNs)**. We are going to build a tiny brain and teach it to tell the difference between a picture of a Circle and a picture of a Square.

The Code

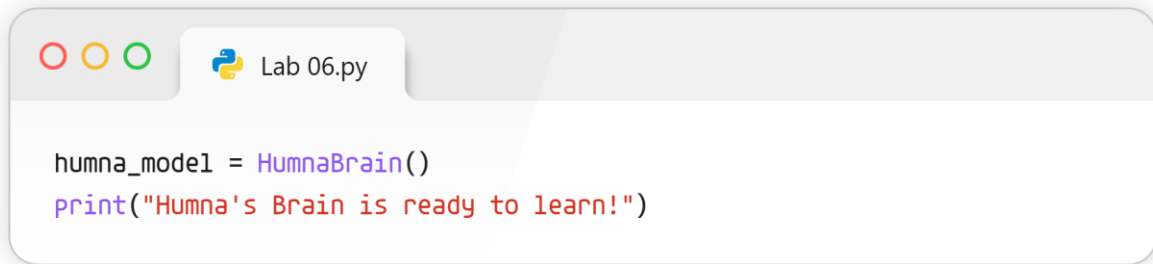


```
import torch
import torch.nn as nn
```



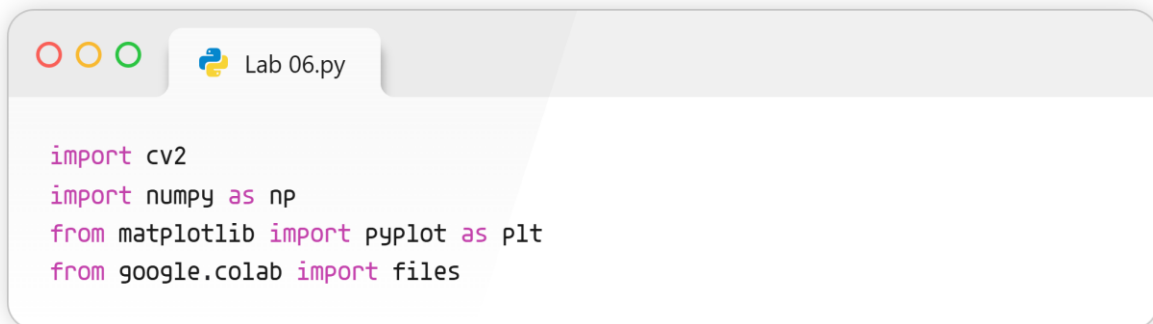
```
class HumnaBrain(nn.Module):
    def __init__(self):
        super().__init__()
        self.humna_eye = nn.Conv2d(1, 8, kernel_size=3)
        self.humna_decision = nn.Linear(8 * 62 * 62, 2)

    def forward(self, humna_x):
        humna_x = self.humna_eye(humna_x)
        humna_x = humna_x.view(humna_x.size(0), -1)
        humna_x = self.humna_decision(humna_x)
        return humna_x
```

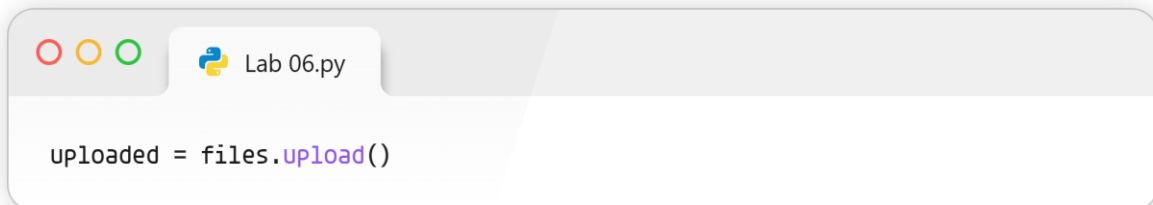


```
humna_model = HumnaBrain()
print("Humna's Brain is ready to learn!")
```

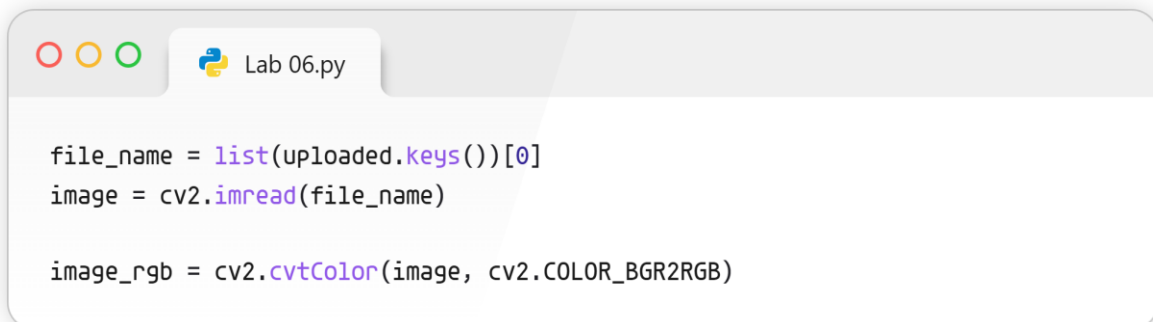
Humna's Brain is ready to learn!



```
import cv2
import numpy as np
from matplotlib import pyplot as plt
from google.colab import files
```



```
uploaded = files.upload()
```



```
file_name = list(uploaded.keys())[0]
image = cv2.imread(file_name)

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
Lab 06.py

sp = 20
sr = 40

shifted_image = cv2.pyrMeanShiftFiltering(image, sp, sr)
shifted_rgb = cv2.cvtColor(shifted_image, cv2.COLOR_BGR2RGB)
```

```
Lab 06.py

plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(shifted_rgb)
plt.title(f"Mean Shift (sp={sp}, sr={sr})")
plt.axis('off')
plt.show()
```

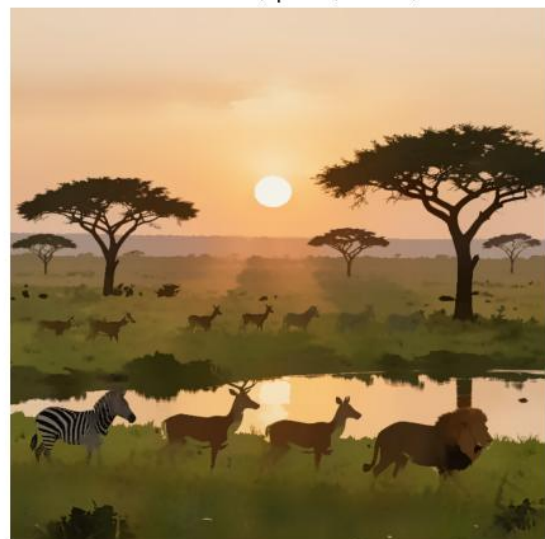
Choose files Wildlife ani...un rises_.jpg

Wildlife animals walking as the sun rises_.jpg(image/jpeg) - 75298 bytes, last modified: 18/06/2026 - 100% done
Saving Wildlife animals walking as the sun rises_.jpg to Wildlife animals walking as the sun rises_.jpg

Original Image



Mean Shift (sp=20, sr=40)



GrayScale Images



Lab 06.py

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```



Lab 06.py

```
threshold_val = 127  
ret, thresh_img = cv2.threshold(gray, threshold_val, 255, cv2.THRESH_BINARY)
```



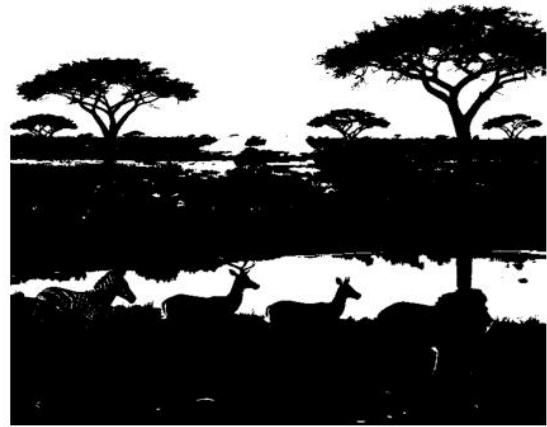
Lab 06.py

```
plt.figure(figsize=(12, 6))  
plt.subplot(1, 2, 1)  
plt.imshow(gray, cmap='gray')  
plt.title("Grayscale Image")  
plt.axis('off')  
  
plt.subplot(1, 2, 2)  
plt.imshow(thresh_img, cmap='gray')  
plt.title(f"Thresholded Image (Val={threshold_val})")  
plt.axis('off')  
plt.show()
```

Grayscale Image



Thresholded Image (Val=127)

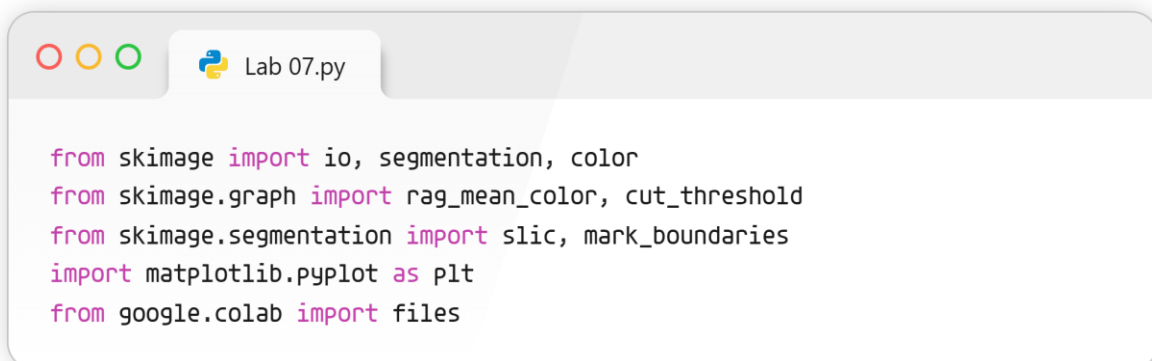


LAB 7: Unsupervised Image Segmentation

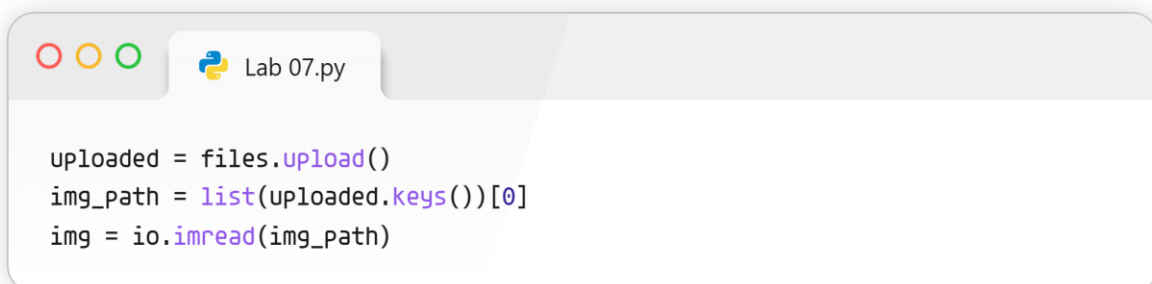
What We Are Doing and Why

THIS LAB introduces an unsupervised image segmentation pipeline that divides an image into meaningful regions without using labeled data. It combines three main techniques: smoothing, superpixel segmentation, and region merging. First, the image is preprocessed using a bilateral filter to reduce noise while preserving edges. Then, the SLIC algorithm is applied to divide the image into small regions called superpixels. These superpixels group similar pixels together based on color and proximity. Next, a Region Adjacency Graph (RAG) is created, where neighboring regions are connected and compared based on color similarity. Similar regions are merged using a threshold, reducing many small segments into fewer meaningful regions. Finally, the results are displayed by comparing the original image, the segmented superpixels, and the final merged output. This lab helps students understand how images can be segmented using clustering and graph-based methods, forming a foundation for advanced image analysis tasks.

The Code

A code editor window titled 'Lab 07.py' with a Python icon. It contains the following Python code:

```
from skimage import io, segmentation, color
from skimage.graph import rag_mean_color, cut_threshold
from skimage.segmentation import slic, mark_boundaries
import matplotlib.pyplot as plt
from google.colab import files
```

A code editor window titled 'Lab 07.py' with a Python icon. It contains the following Python code:

```
uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = io.imread(img_path)
```

```
Lab 07.py

labels = slic(img, n_segments=200, compactness=10, start_label=0)
rag = rag_mean_color(img, labels)
labels_merged = cut_threshold(labels, rag, 35)
```

Choose files 17 of the Most Challenging Jigsaw Puzzles You Can Buy.jpg

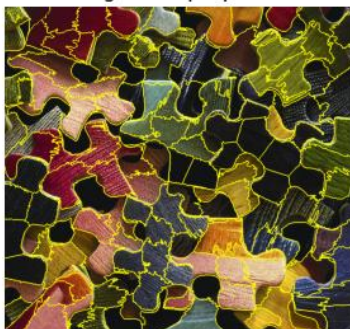
17 of the Most Challenging Jigsaw Puzzles You Can Buy.jpg(image/jpeg) - 120564 bytes, last modified: 18/06/2026 - 100% done

Saving 17 of the Most Challenging Jigsaw Puzzles You Can Buy.jpg to 17 of the Most Challenging Jigsaw Puzzles You Can Buy.jpg

```
Lab 07.py

out = color.label2rgb(labels_merged, img, kind='overlay')
plt.figure(figsize=(12,6))
plt.subplot(1,3,1)
plt.imshow(mark_boundaries(img, labels))
plt.title("Original Superpixels")
plt.axis("off")
plt.subplot(1,3,2)
plt.imshow(mark_boundaries(out, labels_merged))
plt.title("RAG + Merged Segments")
plt.axis("off")
plt.subplot(1,3,3)
plt.imshow(img)
plt.title("real img")
plt.axis("off")
```

Original Superpixels



RAG + Merged Segments



real img

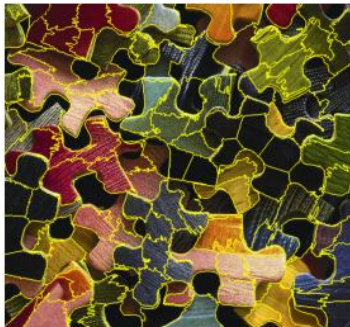




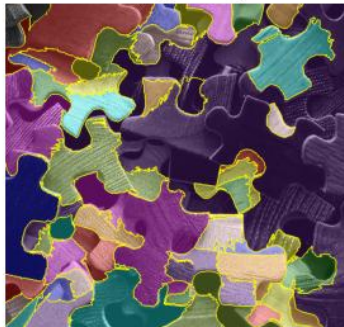
Lab 07.py

```
rag = rag_mean_color(img, labels)
thresh = 35
labels_merged = cut_threshold(labels, rag, thresh)
out = color.label2rgb(labels_merged, img, kind='avg')
plt.figure(figsize=(15, 8))
plt.subplot(1, 2, 1)
plt.imshow(mark_boundaries(img, labels))
plt.title(f"SLIC Superpixels (n=200)")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(mark_boundaries(out, labels_merged))
plt.title(f"Merged Regions (Threshold={thresh})")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Original Superpixels



RAG + Merged Segments



real img



Lab 07.py

```
import matplotlib.pyplot as plt
from skimage import io, segmentation, color, graph
from skimage.segmentation import slic, mark_boundaries
from skimage.graph import rag_mean_color, cut_threshold
img_path = list(uploaded.keys())[0]
img = io.imread(img_path)
labels = slic(img, n_segments=200, compactness=10, start_label=1)
g = rag_mean_color(img, labels)
thresh = 35
labels_merged = cut_threshold(labels, g, thresh)
out_merged = color.label2rgb(labels_merged, img, kind='avg')
```



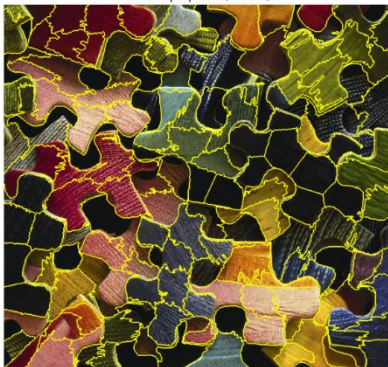

Lab 07.py

```
plt.figure(figsize=(18, 6))
plt.subplot(1, 3, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(mark_boundaries(img, labels))
plt.title("SLIC Superpixels (n=200)")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(mark_boundaries(out_merged, labels_merged))
plt.title(f"RAG Merged (Threshold={thresh})")
plt.axis("off")
plt.tight_layout()
plt.show()
```

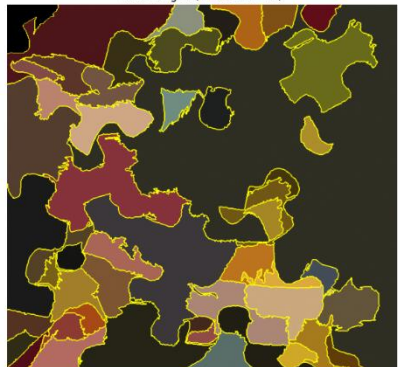
Original Image



SLIC Superpixels (n=200)



RAG Merged (Threshold=35)



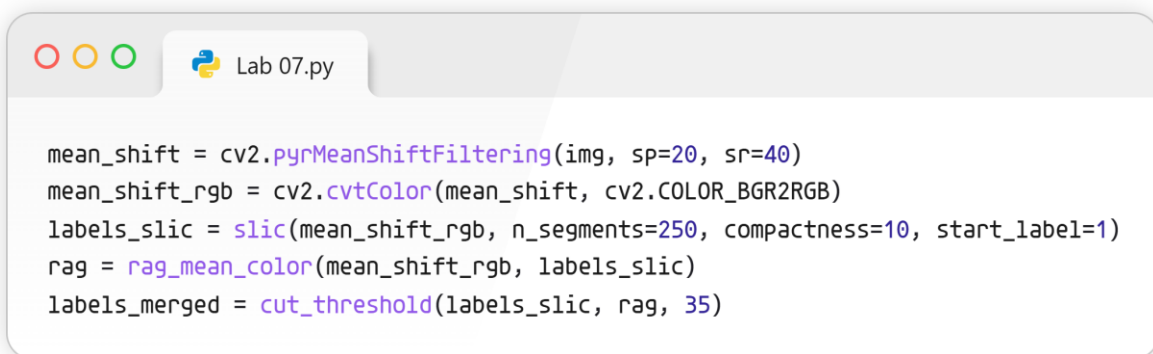
Lab 07.py

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import io, segmentation, color
from skimage.graph import rag_mean_color, cut_threshold
from skimage.segmentation import slic, watershed, mark_boundaries
from scipy import ndimage as ndi
from google.colab import files
```

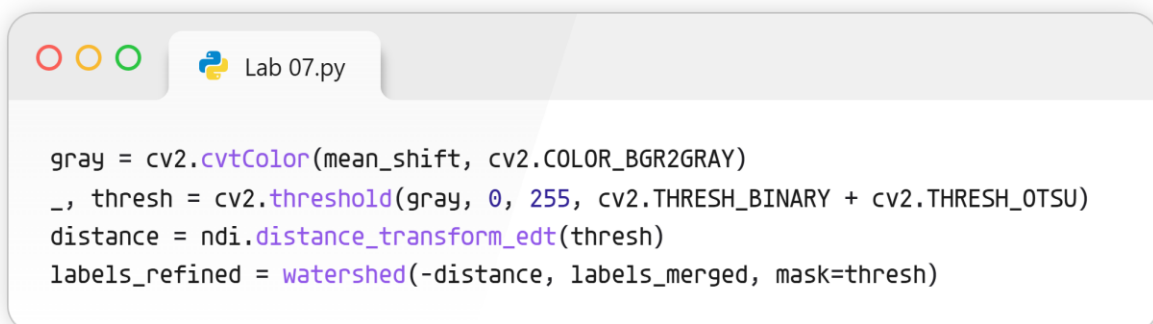


```
uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = cv2.imread(img_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Choose files download (8).jpg
download (8).jpg(image/jpeg) - 130411 bytes, last modified: 18/06/2026 - 100% done
Saving download (8).jpg to download (8).jpg



```
mean_shift = cv2.pyrMeanShiftFiltering(img, sp=20, sr=40)
mean_shift_rgb = cv2.cvtColor(mean_shift, cv2.COLOR_BGR2RGB)
labels_slic = slic(mean_shift_rgb, n_segments=250, compactness=10, start_label=1)
rag = rag_mean_color(mean_shift_rgb, labels_slic)
labels_merged = cut_threshold(labels_slic, rag, 35)
```

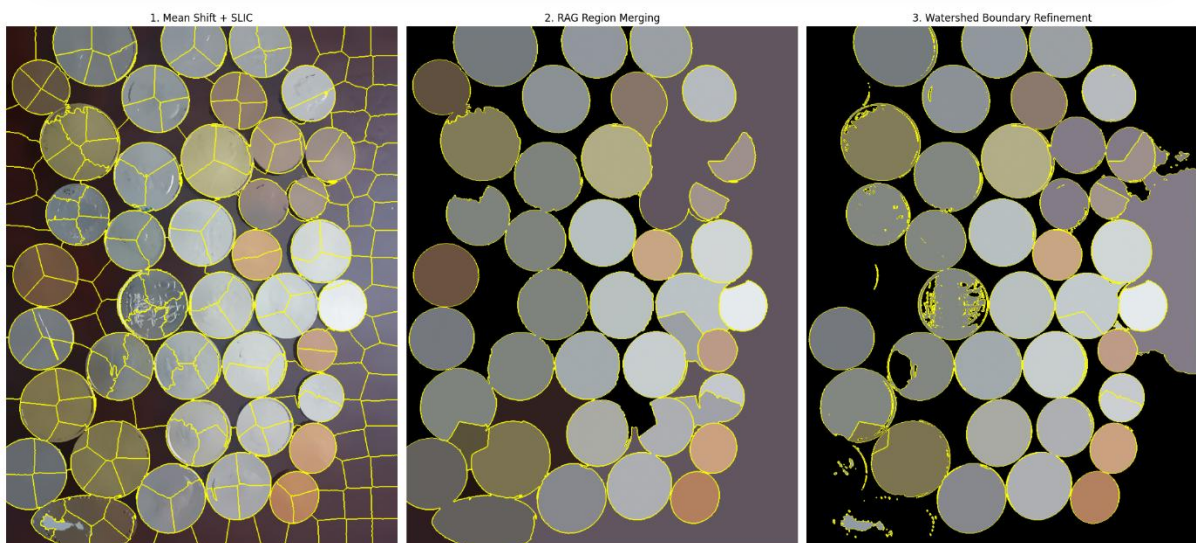


```
gray = cv2.cvtColor(mean_shift, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
distance = ndi.distance_transform_edt(thresh)
labels_refined = watershed(-distance, labels_merged, mask=thresh)
```



```
Lab 07.py

out_initial = mark_boundaries(mean_shift_rgb, labels_slic)
out_merged = color.label2rgb(labels_merged, img_rgb, kind='avg')
out_refined = color.label2rgb(labels_refined, img_rgb, kind='avg')
plt.figure(figsize=(20, 10))
plt.subplot(1, 3, 1)
plt.imshow(out_initial)
plt.title("1. Mean Shift + SLIC")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(mark_boundaries(out_merged, labels_merged))
plt.title("2. RAG Region Merging")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(mark_boundaries(out_refined, labels_refined))
plt.title("3. Watershed Boundary Refinement")
plt.axis("off")
plt.tight_layout()
plt.show()
```



Summary

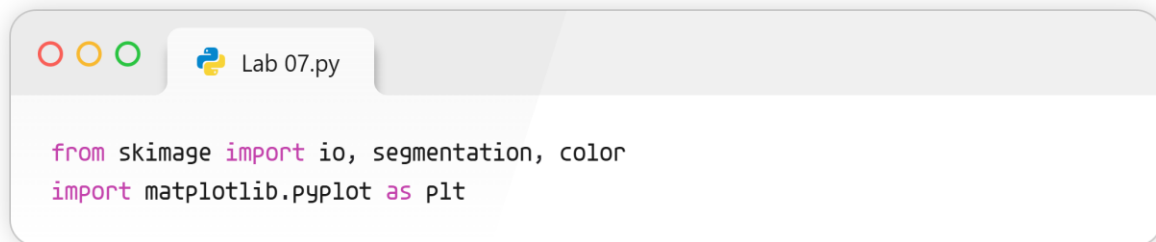
The computer looked for areas where the color was mostly the same and grouped them into chunks. Then, it calculated the average color of each chunk and painted the whole piece that one solid color. This makes it much easier for the computer to understand where objects start and end!

Lab 8: Felzenszwalb Graph-Based Segmentation

What We Are Doing and Why

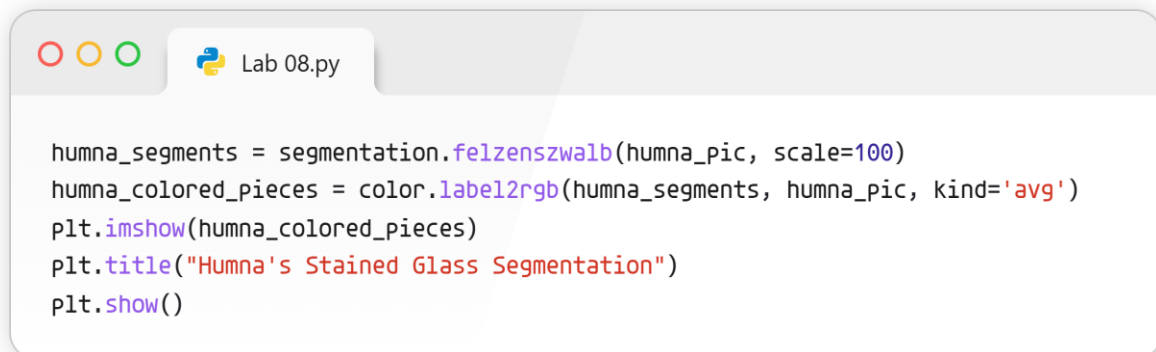
Imagine a stained glass window. It is made of many different pieces of colored glass stuck together. **Image Segmentation** does exactly this to a picture! It groups pixels that have similar colors and textures together into "segments" (pieces). We will use a tool called the **Felzenszwalb Algorithm** to do this.

The Code

A code editor window titled "Lab 07.py" with a Python icon. It contains two lines of Python code:

```
from skimage import io, segmentation, color
import matplotlib.pyplot as plt
```

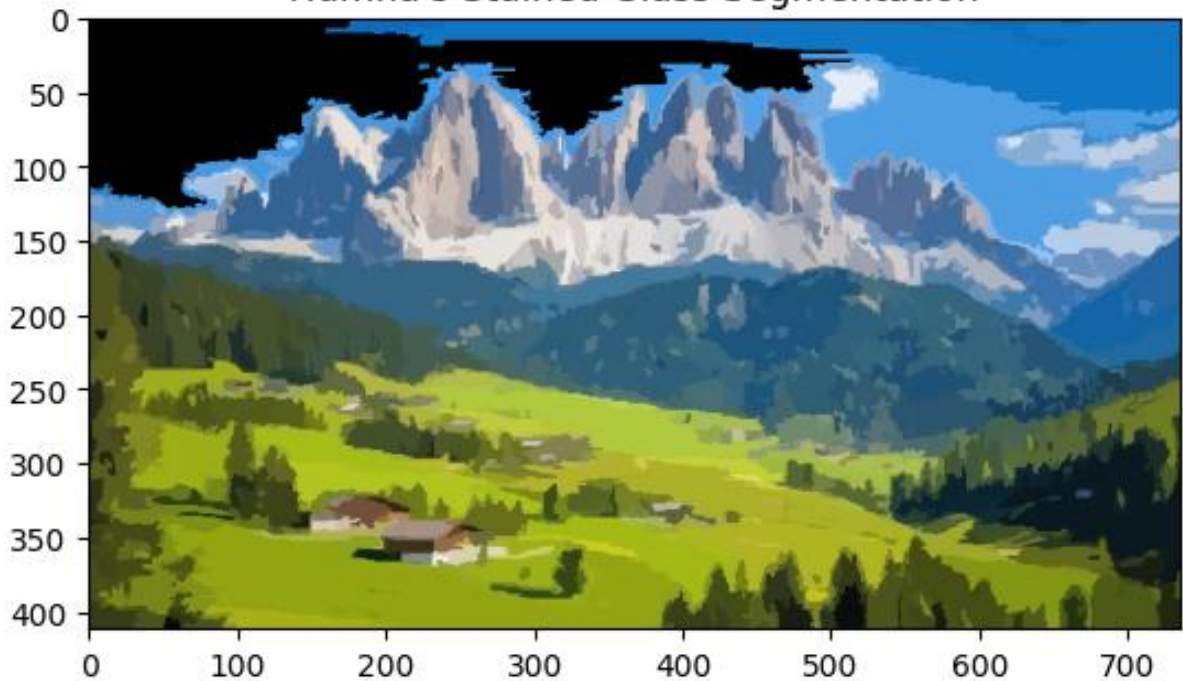
```
from skimage import io, segmentation, color
import matplotlib.pyplot as plt
```

A code editor window titled "Lab 08.py" with a Python icon. It contains five lines of Python code:

```
humna_segments = segmentation.felzenszwalb(humna_pic, scale=100)
humna_colored_pieces = color.label2rgb(humna_segments, humna_pic, kind='avg')
plt.imshow(humna_colored_pieces)
plt.title("Humna's Stained Glass Segmentation")
plt.show()
```

```
humna_segments = segmentation.felzenszwalb(humna_pic, scale=100)
humna_colored_pieces = color.label2rgb(humna_segments, humna_pic, kind='avg')
plt.imshow(humna_colored_pieces)
plt.title("Humna's Stained Glass Segmentation")
plt.show()
```

Humna's Stained Glass Segmentation



Lab 08.py

```
from skimage import io, segmentation, color
from skimage.graph import rag_mean_color, cut_threshold
from skimage.segmentation import slic, mark_boundaries
import matplotlib.pyplot as plt
from google.colab import files
uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = io.imread(img_path)
labels = slic(img, n_segments=200, compactness=10, start_label=0)
rag = rag_mean_color(img, labels)
labels_merged = cut_threshold(labels, rag, 35)
```

[Choose files](#) 17 of the M... Can Buy.jpg

17 of the Most Challenging Jigsaw Puzzles You Can Buy.jpg(image/jpeg) - 120564 bytes, last modified: 18/06/2026 - 100% done

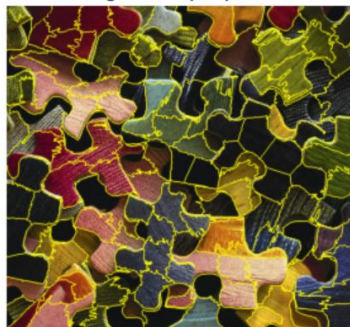
Saving 17 of the Most Challenging Jigsaw Puzzles You Can Buy.jpg to 17 of the Most Challenging Jigsaw Puzzles You Can Buy (1).jpg

```
Lab 08.py

out = color.label2rgb(labels_merged, img, kind='overlay')
plt.figure(figsize=(12,6))
plt.subplot(1,3,1)
plt.imshow(mark_boundaries(img, labels))
plt.title("Original Superpixels")
plt.axis("off")
plt.subplot(1,3,2)
plt.imshow(mark_boundaries(out, labels_merged))
plt.title("RAG + Merged Segments")
plt.axis("off")
plt.subplot(1,3,3)
plt.imshow(img)
plt.title("real img")
plt.axis("off")
```

(np.float64(-0.5), np.float64(735.5), np.float64(687.5), np.float64(-0.5))

Original Superpixels



RAG + Merged Segments

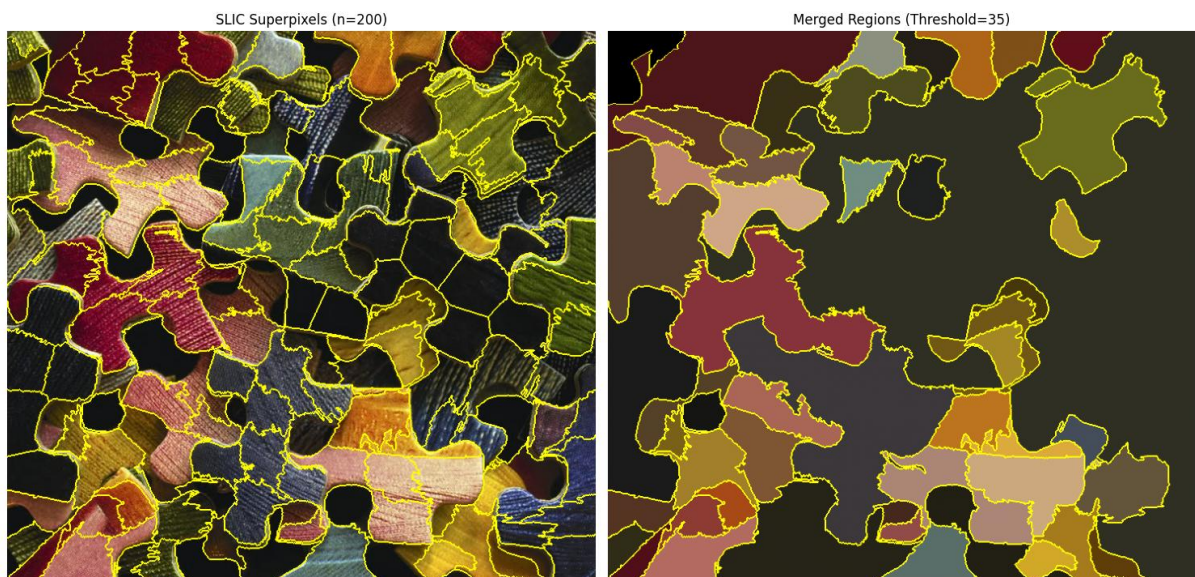


real img




```
Lab 08.py

rag = rag_mean_color(img, labels)
thresh = 35
labels_merged = cut_threshold(labels, rag, thresh)
out = color.label2rgb(labels_merged, img, kind='avg')
plt.figure(figsize=(15, 8))
plt.subplot(1, 2, 1)
plt.imshow(mark_boundaries(img, labels))
plt.title(f"SLIC Superpixels (n=200)")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(mark_boundaries(out, labels_merged))
plt.title(f"Merged Regions (Threshold={thresh})")
plt.axis("off")
plt.tight_layout()
plt.show()
```

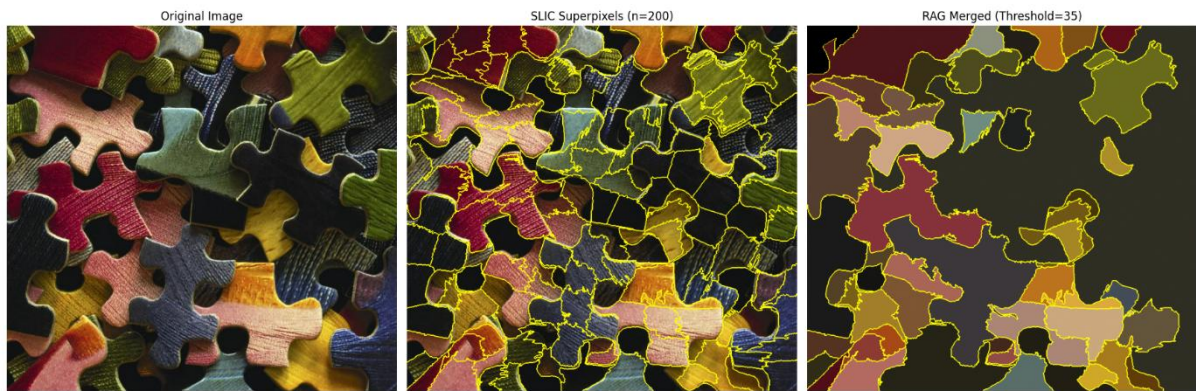


```
Lab 08.py

import matplotlib.pyplot as plt
from skimage import io, segmentation, color, graph
from skimage.segmentation import slic, mark_boundaries
from skimage.graph import rag_mean_color, cut_threshold
img_path = list(uploaded.keys())[0]
img = io.imread(img_path)
labels = slic(img, n_segments=200, compactness=10, start_label=1)
g = rag_mean_color(img, labels)
thresh = 35
labels_merged = cut_threshold(labels, g, thresh)
out_merged = color.label2rgb(labels_merged, img, kind='avg')
```

```
Lab 08.py

plt.figure(figsize=(18, 6))
plt.subplot(1, 3, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(mark_boundaries(img, labels))
plt.title("SLIC Superpixels (n=200)")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(mark_boundaries(out_merged, labels_merged))
plt.title(f"RAG Merged (Threshold={thresh})")
plt.axis("off")
plt.tight_layout()
plt.show()
```



```

import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import io, segmentation, color
from skimage.graph import rag_mean_color, cut_threshold
from skimage.segmentation import slic, watershed, mark_boundaries
from scipy import ndimage as ndi
from google.colab import files

```

```

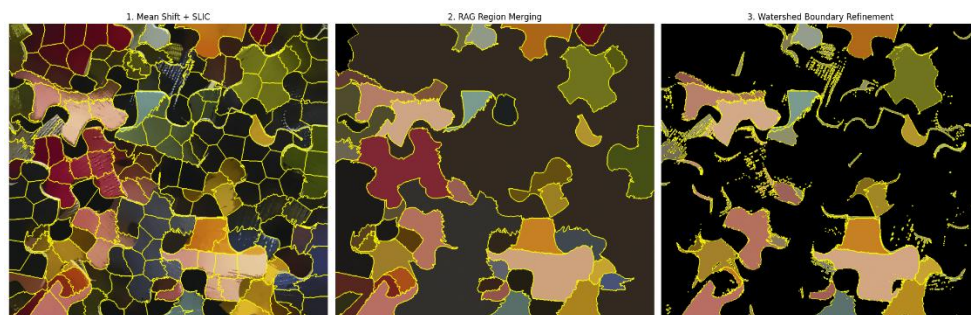
uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = cv2.imread(img_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

```

Choose files 17 of the M... Can Buy.jpg
17 of the Most Challenging Jigsaw Puzzles You Can Buy.jpg(image/jpeg) - 120564 bytes, last modified: 18/06/2026 - 100% done
 Saving 17 of the Most Challenging Jigsaw Puzzles You Can Buy.jpg to 17 of the Most


```
mean_shift = cv2.pyrMeanShiftFiltering(img, sp=20, sr=40)
mean_shift_rgb = cv2.cvtColor(mean_shift, cv2.COLOR_BGR2RGB)
labels_slic = slic(mean_shift_rgb, n_segments=250, compactness=10, start_label=1)
rag = rag_mean_color(mean_shift_rgb, labels_slic)
labels_merged = cut_threshold(labels_slic, rag, 35)
```

```
gray = cv2.cvtColor(mean_shift, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
distance = ndi.distance_transform_edt(thresh)
labels_refined = watershed(-distance, labels_merged, mask=thresh)
out_initial = mark_boundaries(mean_shift_rgb, labels_slic)
out_merged = color.label2rgb(labels_merged, img_rgb, kind='avg')
out_refined = color.label2rgb(labels_refined, img_rgb, kind='avg')
plt.figure(figsize=(20, 10))
plt.subplot(1, 3, 1)
plt.imshow(out_initial)
plt.title("1. Mean Shift + SLIC")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(mark_boundaries(out_merged, labels_merged))
plt.title("2. RAG Region Merging")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(mark_boundaries(out_refined, labels_refined))
plt.title("3. Watershed Boundary Refinement")
plt.axis("off")
plt.tight_layout()
plt.show()
```



What We Are Doing and Why

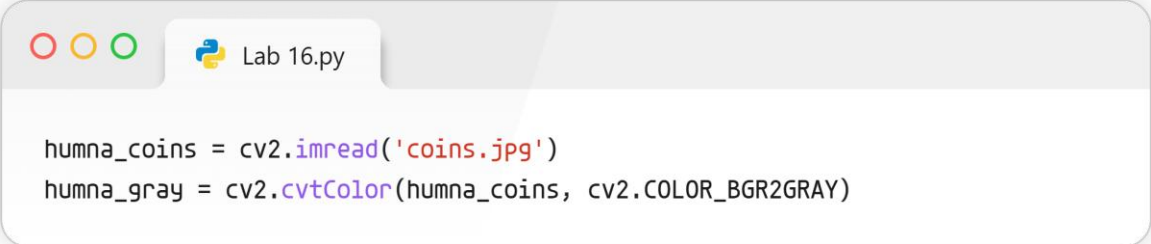
Sometimes objects in a picture are touching each other, like two coins sitting side-by-side. The computer might think they are just one giant object. We fix this using the **Watershed** trick.

Imagine the picture is a 3D mountain landscape. The bright spots are hills and the dark spots are valleys. If we slowly pour water into the valleys, eventually two pools of water will meet. Right where they meet, we build a "dam" to separate them. This perfectly outlines touching objects!

The Code



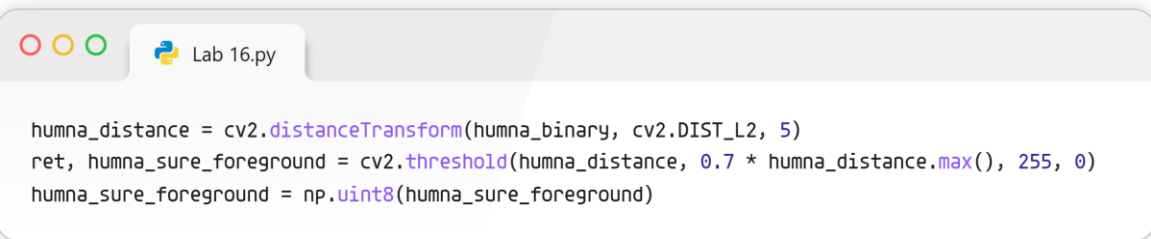
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```



```
humna_coins = cv2.imread('coins.jpg')
humna_gray = cv2.cvtColor(humna_coins, cv2.COLOR_BGR2GRAY)
```



```
ret, humna_binary = cv2.threshold(humna_gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
```



```
humna_distance = cv2.distanceTransform(humna_binary, cv2.DIST_L2, 5)
ret, humna_sure_foreground = cv2.threshold(humna_distance, 0.7 * humna_distance.max(), 255, 0)
humna_sure_foreground = np.uint8(humna_sure_foreground)
```



```
ret, humna_markers = cv2.connectedComponents(humna_sure_foreground)
```



Lab 16.py

```
humna_markers = humna_markers + 1  
cv2.watershed(humna_coins, humna_markers)
```



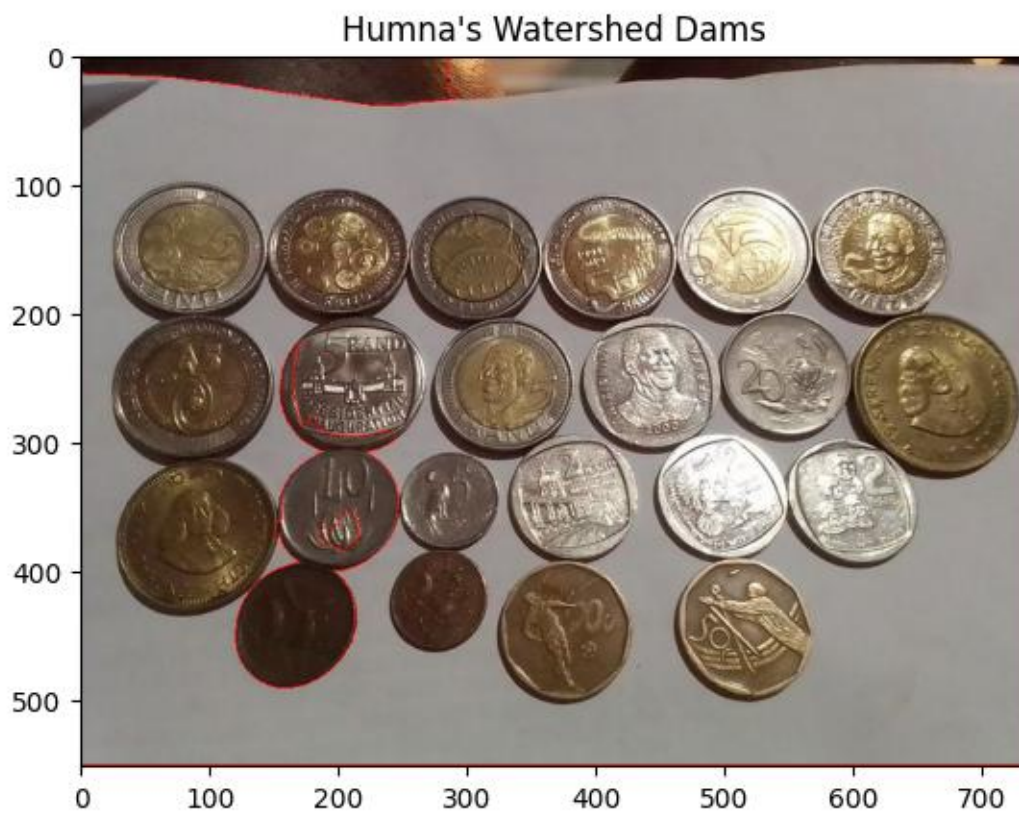
Lab 16.py

```
humna_coins[humna_markers == -1] = [0, 0, 255]
```



Lab 16.py

```
plt.imshow(cv2.cvtColor(humna_coins, cv2.COLOR_BGR2RGB))  
plt.title("Humna's Watershed Dams")  
plt.show()
```



Summary

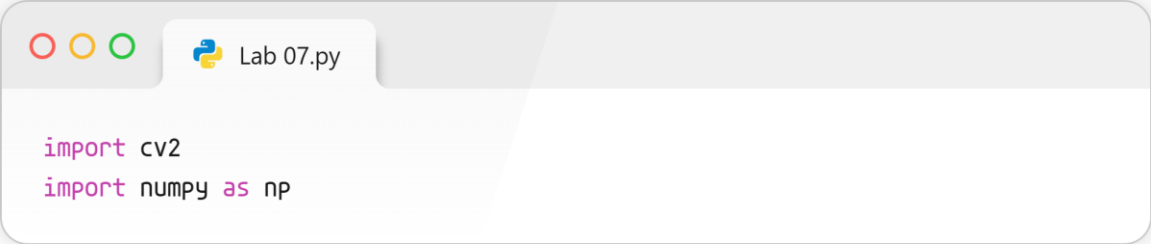
First, we made the picture completely black and white. Then, we searched for the very deepest, center part of each object to act as the "starting puddle." When we applied watershed(), the computer flooded the image from those puddles and drew bright red boundaries exactly where the objects touched each other

Lab 9: Image Segmentation Pipeline

What We Are Doing and Why

Now for the coolest part! Object Detection doesn't just say "this is a picture of a dog." It draws a box exactly *where* the dog is! We will use a pre-trained brain called **MobileNet SSD**. It is very fast, so it can look at a webcam video in real-time and find people, cars, birds, and more.

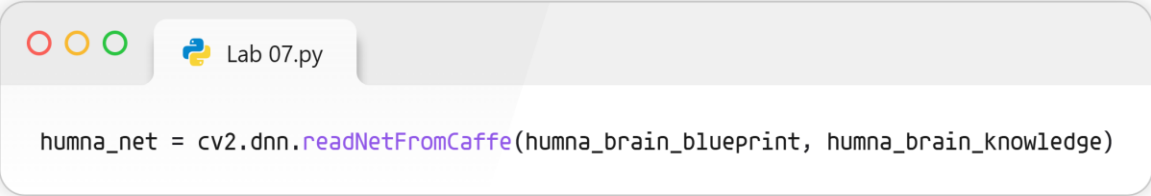
The Code

A code editor window titled "Lab 07.py" with a Python icon. It contains two lines of code: `import cv2` and `import numpy as np`.

```
import cv2
import numpy as np
```

A code editor window titled "Lab 07.py" with a Python icon. It contains two lines of code: `humna_brain_blueprint = "MobileNetSSD_deploy.prototxt"` and `humna_brain_knowledge = "MobileNetSSD_deploy.caffemodel"`.

```
humna_brain_blueprint = "MobileNetSSD_deploy.prototxt"
humna_brain_knowledge = "MobileNetSSD_deploy.caffemodel"
```

A code editor window titled "Lab 07.py" with a Python icon. It contains one line of code: `humna_net = cv2.dnn.readNetFromCaffe(humna_brain_blueprint, humna_brain_knowledge)`.

```
humna_net = cv2.dnn.readNetFromCaffe(humna_brain_blueprint, humna_brain_knowledge)
```

A code editor window titled "Lab 07.py" with a Python icon. It contains one line of code: `humna_classes = ["background", "aeroplane", "bicycle", "bird", "boat", "bottle", "bus", "car", "cat", "chair", "cow", "diningtable", "dog", "horse", "motorbike", "person", "pottedplant", "sheep", "sofa", "train", "tvmonitor"]`.

```
humna_classes = ["background", "aeroplane", "bicycle", "bird", "boat",
                  "bottle", "bus", "car", "cat", "chair", "cow", "diningtable",
                  "dog", "horse", "motorbike", "person", "pottedplant",
                  "sheep", "sofa", "train", "tvmonitor"]
```



Lab 07.py

```
humna_frame = cv2.imread("street.jpg")
h, w = humna_frame.shape[:2]
```



Lab 07.py

```
humna_blob = cv2.dnn.blobFromImage(cv2.resize(humna_frame, (300, 300)), 0.007843, (300, 300), 127.5)
```



Lab 07.py

```
humna_net.setInput(humna_blob)
humna_detections = humna_net.forward()
```



Lab 07.py

```
for i in range(humna_detections.shape[2]):
    humna_confidence = humna_detections[0, 0, i, 2]
    if humna_confidence > 0.5:
        humna_id = int(humna_detections[0, 0, i, 1])
        humna_box = humna_detections[0, 0, i, 3:7] * np.array([w, h, w, h])
        (startX, startY, endX, endY) = humna_box.astype("int")
        humna_label = humna_classes[humna_id]
        cv2.rectangle(humna_frame, (startX, startY), (endX, endY), (0, 255, 0), 2)
        cv2.putText(humna_frame, humna_label, (startX, startY - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
```



Lab 07.py

```
cv2.imshow("Humna's Object Detector", humna_frame)
cv2.waitKey(0)
```



```

File 'ssd_mobilenet_v1_coco_11_06_2017.tar.gz' already there; not retrieving.
ssd_mobilenet_v1_coco_11_06_2017/
ssd_mobilenet_v1_coco_11_06_2017/model.ckpt.index
ssd_mobilenet_v1_coco_11_06_2017/model.ckpt.meta
ssd_mobilenet_v1_coco_11_06_2017/frozen_inference_graph.pb
ssd_mobilenet_v1_coco_11_06_2017/model.ckpt.data-00000-of-00001
ssd_mobilenet_v1_coco_11_06_2017/graph.pbtxt
--2026-06-18 11:13:19-- https://raw.githubusercontent.com/opencv/opencv\_extra/master/testdata/dnn/ssd\_mobilenet\_v1\_coco.pbtxt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 61355 (60K) [text/plain]
Saving to: 'ssd_mobilenet_v1_coco.pbtxt'

ssd_mobilenet_v1_co 100%[=====] 59.92K --.-KB/s in 0.01s

2026-06-18 11:13:19 (5.07 MB/s) - 'ssd_mobilenet_v1_coco.pbtxt' saved [61355/61355]

File 'street.jpg' already there; not retrieving.

```



```
from skimage import io, segmentation, color, filters, morphology
from skimage.segmentation import mark_boundaries, watershed
from skimage.restoration import denoise_bilateral
from skimage.feature import canny
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import numpy as np
from scipy import ndimage as ndi
from google.colab import files
uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = io.imread(img_path)
if img.ndim == 2:
    img = np.stack([img] * 3, axis=-1)
if img.shape[2] == 4:
    img = img[:, :, :3]
print(f"Image loaded: {img.shape} dtype={img.dtype}")
img_float = img.astype(np.float64) / 255.0
img_smoothed = denoise_bilateral(
```

```
img_smoothed = denoise_bilateral(
    img_float,
    sigma_color=0.1,
    sigma_spatial=5,
    channel_axis=-1
)
```



Lab 09.py

```
img_ms = (img_smoothed * 255).clip(0, 255).astype(np.uint8)
print("Mean Shift smoothing complete.")
seg_felz = segmentation.felzenszwalb(img_ms, scale=150, sigma=0.5, min_size=60)
n_felz = len(np.unique(seg_felz))
print(f"Felzenszwalb: {n_felz} segments (used as Watershed markers)")
gray = color.rgb2gray(img_ms)
gradient = filters.sobel(gray)
markers = ndi.label(seg_felz)[0]
seg_ws = watershed(gradient, markers=markers, compactness=0.001)
n_ws = len(np.unique(seg_ws))
print(f"Watershed: {n_ws} refined regions")
felz_colored = color.label2rgb(seg_felz, img_ms, kind='avg')
ws_colored = color.label2rgb(seg_ws, img_ms, kind='avg')
```



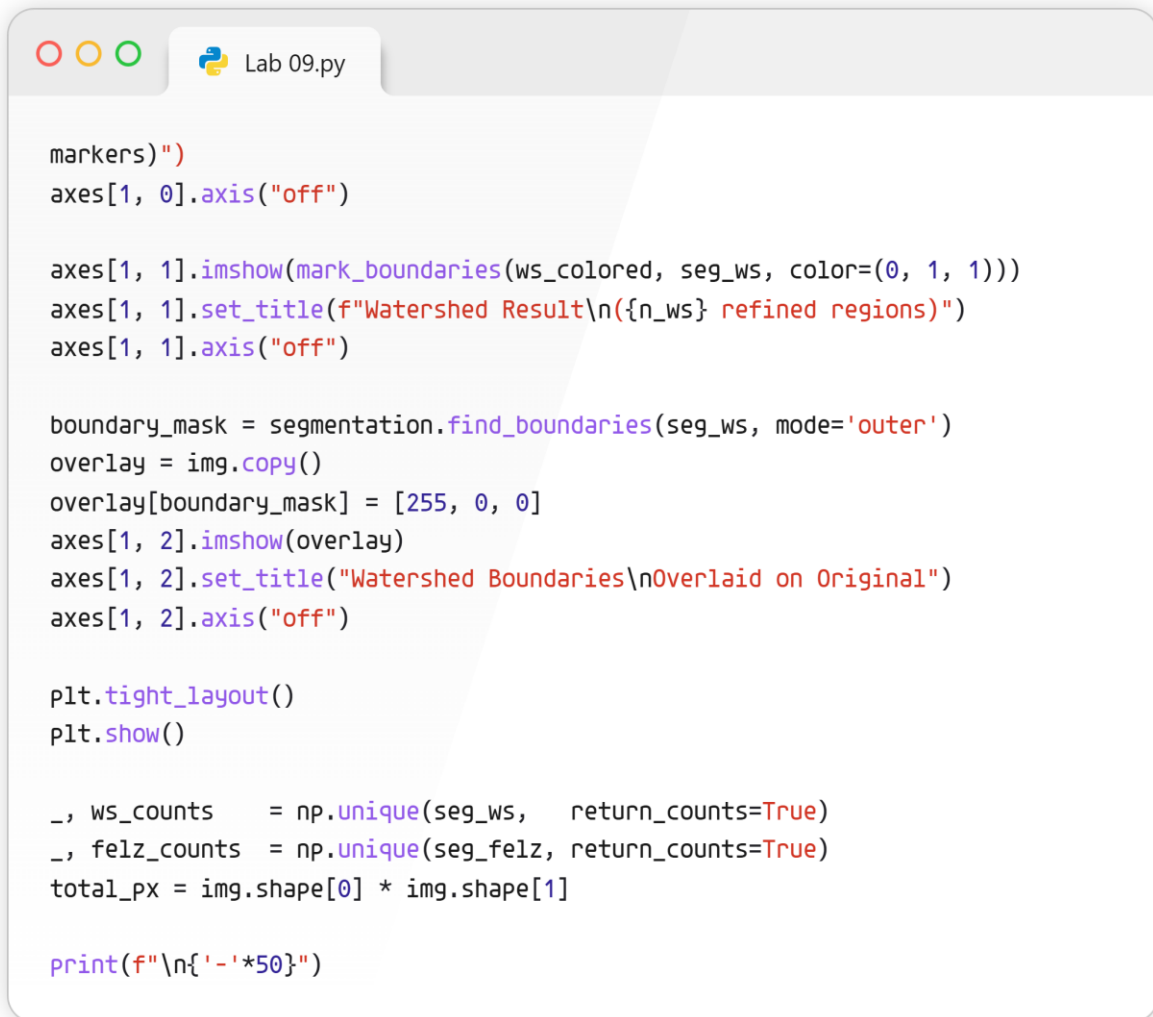
```
fig, axes = plt.subplots(2, 3, figsize=(18, 11))
fig.suptitle(
    "Task 3 - Mean Shift → Felzenszwalb → Watershed\n"
    "Refining boundaries & separating touching objects",
    fontsize=13, fontweight='bold'
)

axes[0, 0].imshow(img)
axes[0, 0].set_title("Original Image")
axes[0, 0].axis("off")

axes[0, 1].imshow(img_ms)
axes[0, 1].set_title("After Mean Shift\n(noise reduced, colors smoothed)")
axes[0, 1].axis("off")

axes[0, 2].imshow(gradient, cmap='gray')
axes[0, 2].set_title("Sobel Gradient\n(Watershed elevation surface)")
axes[0, 2].axis("off")

axes[1, 0].imshow(mark_boundaries(felz_colored, seg_felz, color=(1, 0.5, 0)))
axes[1, 0].set_title(f"Felzenszwalb Segments\n({n_felz} regions → Watershed")
```



```
markers)")
axes[1, 0].axis("off")

axes[1, 1].imshow(mark_boundaries(ws_colored, seg_ws, color=(0, 1, 1)))
axes[1, 1].set_title(f"Watershed Result\n({n_ws} refined regions)")
axes[1, 1].axis("off")

boundary_mask = segmentation.find_boundaries(seg_ws, mode='outer')
overlay = img.copy()
overlay[boundary_mask] = [255, 0, 0]
axes[1, 2].imshow(overlay)
axes[1, 2].set_title("Watershed Boundaries\nOverlaid on Original")
axes[1, 2].axis("off")

plt.tight_layout()
plt.show()

_, ws_counts = np.unique(seg_ws, return_counts=True)
_, felz_counts = np.unique(seg_felz, return_counts=True)
total_px = img.shape[0] * img.shape[1]

print(f"\n{'-'*50}")
```

Choose files 3542.avif

3542.avif(image/avif) - 53778 bytes, last modified: 18/06/2026 - 100% done

Saving 3542.avif to 3542 (2).avif

Image loaded: (992, 1240, 3) dtype=uint8

Mean Shift smoothing complete.

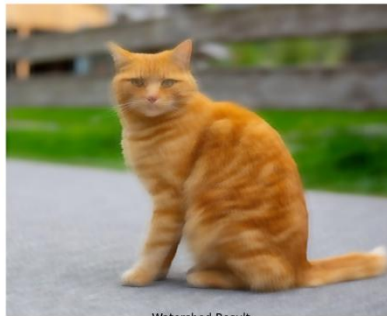
Felzenszwalb: 377 segments (used as Watershed markers)

Watershed: 3 refined regions

Task 3 - Mean Shift → Felzenszwalb → Watershed
Refining boundaries & separating touching objects
After Mean Shift
(noise reduced, colors smoothed)



Original Image



Felzenszwalb Segments
(377 regions → Watershed)



Sobel Gradient
(Watershed elevation surface)



Watershed Result
(3 refined regions)



Watershed Boundaries
Overlaid on Original



Felzenszwalb	377	3262.8
Watershed	3	410026.7

Total pixels: 1,230,080

[Choose files](#) No file chosen

[Cancel upload](#)

Summary

We loaded an AI that already knows how to recognize 20 different items. We prepared our image into a "blob" (the format the AI likes). The AI processed the image and gave us the locations (bounding boxes) of everything it recognized. Finally, we drew green boxes and name tags around those items!

What We Are Doing and Why

Normal AI just says, "There is a cat in this picture." But a **Segmentation AI** acts like a magic marker, it draws a perfectly shaped mask exactly over the cat's body! To do this, it uses an **Encoder** (to shrink the image and understand it) and a **Decoder** (to blow the image back up and color the exact pixels).

The Code



Lab 12.py

```
import tensorflow as tf
from tensorflow.keras import layers, models
```



Lab 12.py

```
humna_segmenter = models.Sequential()
```



Lab 12.py

```
humna_segmenter.add(layers.Conv2D(32, (3,3), activation='relu', padding='same', input_shape=(128,128,3)))
humna_segmenter.add(layers.MaxPooling2D((2,2)))
humna_segmenter.add(layers.Conv2D(64, (3,3), activation='relu', padding='same'))
```



Lab 12.py

```
humna_segmenter.add(layers.UpSampling2D((2,2)))
humna_segmenter.add(layers.Conv2D(3, (3,3), activation='sigmoid', padding='same'))

print("Humna's Segmentation AI blueprint is ready!")
humna_segmenter.summary()
```

Humna's Segmentation AI blueprint is ready!

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning:

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_1 (Conv2D)	(None, 64, 64, 64)	18,496
up_sampling2d (UpSampling2D)	(None, 128, 128, 64)	0
conv2d_2 (Conv2D)	(None, 128, 128, 3)	1,731

Total params: 21,123 (82.51 KB)

Trainable params: 21,123 (82.51 KB)

Non-trainable params: 0 (0.00 B)

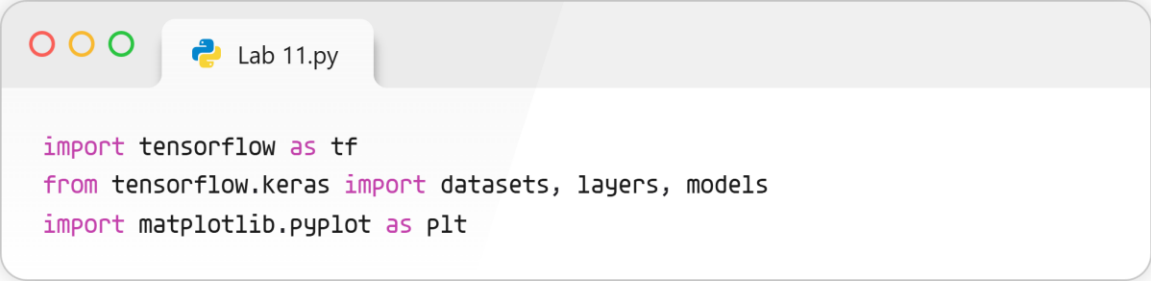
Summary

We built a blueprint for a very special AI. The MaxPooling layers act like a trash compactor, squeezing the image down so the AI can figure out *what* the object is. The UpSampling layers act like a balloon, inflating the image back up so the AI can draw a colorful mask showing *where* the object is!

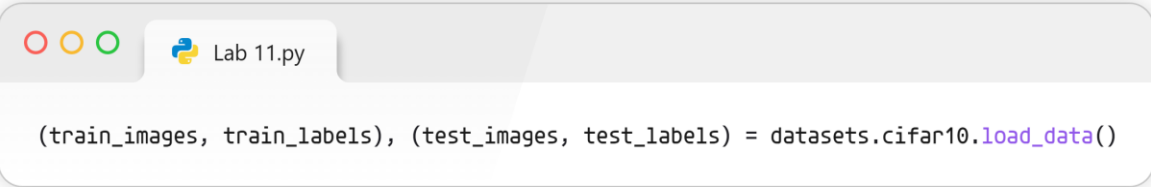
Lab 10: CIFAR-10 Image Classification using CNN

What and why we Are Doing

In this lab, we are designing and training a Convolutional Neural Network (CNN) to classify images from the CIFAR-10 dataset into ten distinct categories. We utilize TensorFlow and Keras to build a model comprising convolutional layers for feature extraction and dense layers for final classification. The process involves loading the data, normalizing pixel values, defining the architecture, and executing the training phase. Finally, we evaluate the model's accuracy on unseen test data and visualize the learning progress to ensure our network is learning effectively.

A code editor window titled 'Lab 11.py' with a Python icon. It contains the following code:

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

A code editor window titled 'Lab 11.py' with a Python icon. It contains the following code:

```
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()
```

A code editor window titled 'Lab 11.py' with a Python icon. It contains the following code:

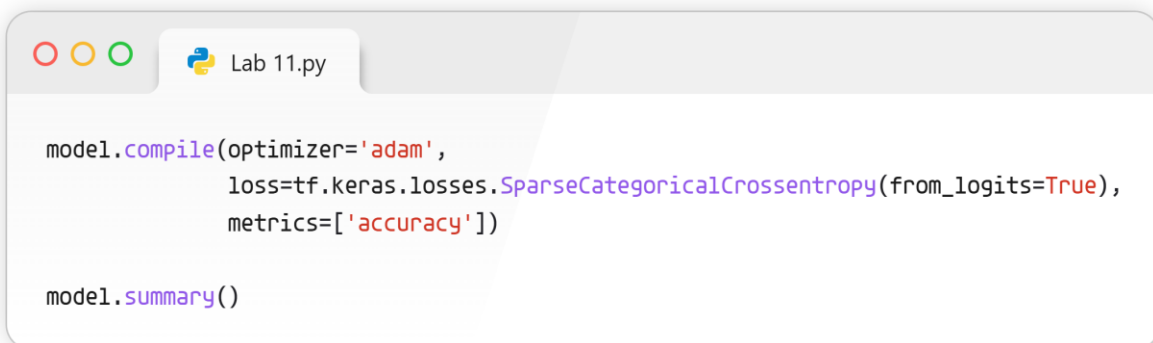
```
train_images, test_images = train_images / 255.0, test_images / 255.0
```

A code editor window titled 'Lab 11.py' with a Python icon. It contains the following code:

```
class_names = ['Airplane', 'Automobile', 'Bird', 'Cat', 'Deer',
               'Dog', 'Frog', 'Horse', 'Ship', 'Truck']
```



```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))
```



```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

model.summary()
```



```
print("\nStarting training...")
history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
print(f"\nTest Accuracy: {test_acc * 100:.2f}%")
```



Lab 11.py

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label = 'Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
plt.title('Training vs Validation Accuracy')
plt.show()
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>

170498071/170498071 — 6s 0us/step

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65,600
dense_1 (Dense)	(None, 10)	650

Total params: 122,570 (478.79 KB)

Trainable params: 122,570 (478.79 KB)

Non-trainable params: 0 (0.00 B)

Starting training...

Epoch 1/10

1563/1563 — 23s 14ms/step - accuracy: 0.4376 - loss: 1.5357 - val_accuracy: 0.5527 - val_loss: 1.2405

Epoch 2/10

1563/1563 — 22s 14ms/step - accuracy: 0.5870 - loss: 1.1668 - val_accuracy: 0.6211 - val_loss: 1.0697

Epoch 3/10

1563/1563 — 22s 14ms/step - accuracy: 0.6439 - loss: 1.0106 - val_accuracy: 0.6376 - val_loss: 1.0510

Epoch 4/10

1563/1563 — 22s 14ms/step - accuracy: 0.6778 - loss: 0.9207 - val_accuracy: 0.6850 - val_loss: 0.9084

Epoch 5/10

1563/1563 — 22s 14ms/step - accuracy: 0.7033 - loss: 0.8496 - val_accuracy: 0.6817 - val_loss: 0.9176

Epoch 6/10

1563/1563 — 22s 14ms/step - accuracy: 0.7214 - loss: 0.7952 - val_accuracy: 0.7091 - val_loss: 0.8725

Epoch 7/10

1563/1563 — 22s 14ms/step - accuracy: 0.7392 - loss: 0.7456 - val_accuracy: 0.7050 - val_loss: 0.8638

Epoch 8/10

1563/1563 — 22s 14ms/step - accuracy: 0.7523 - loss: 0.7031 - val_accuracy: 0.7038 - val_loss: 0.8720

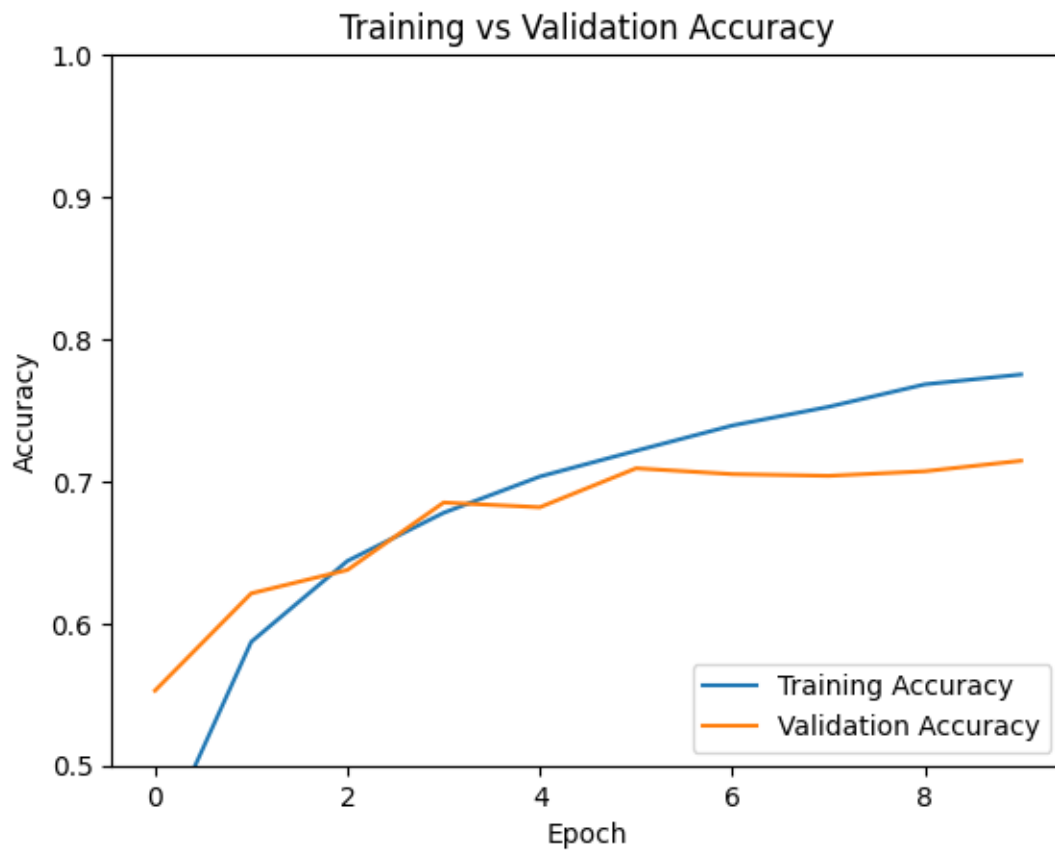
Epoch 9/10

1563/1563 — 22s 14ms/step - accuracy: 0.7682 - loss: 0.6679 - val_accuracy: 0.7070 - val_loss: 0.8626

Epoch 10/10

1563/1563 — 22s 14ms/step - accuracy: 0.7751 - loss: 0.6398 - val_accuracy: 0.7144 - val_loss: 0.8346

313/313 - 1s - 4ms/step - accuracy: 0.7144 - loss: 0.8346



Summary

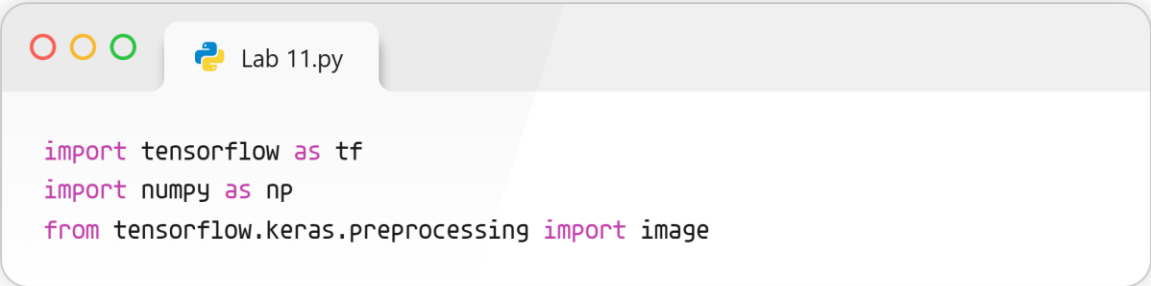
To summarize, Lab 10 provides a hands-on introduction to image classification by guiding us through the complete pipeline of building a CNN. We learned how to preprocess image data, construct a multi-layered neural network architecture, and evaluate its predictive performance on the CIFAR-10 dataset. This practical experience solidifies our understanding of how convolutional filters extract meaningful features from raw image pixels. By the end of the exercise, we successfully created a functional model capable of categorizing everyday objects with measurable accuracy.

Lab 11: Image Classification using Pre-trained CNN (MobileNetV2)

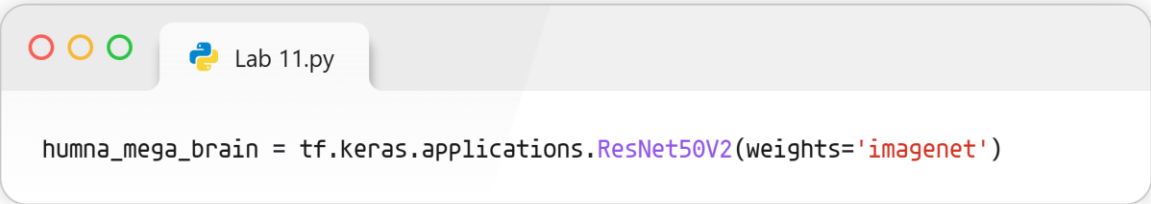
What We Are Doing and Why

Training an AI from scratch takes weeks and requires supercomputers. Luckily, smart scientists have already trained "Mega Brains" on millions of pictures and shared them with us! We will use a pre-trained brain called **ResNet50V2** to classify a picture.

The Code

A code editor window titled "Lab 11.py" with a Python icon. It contains the following code:

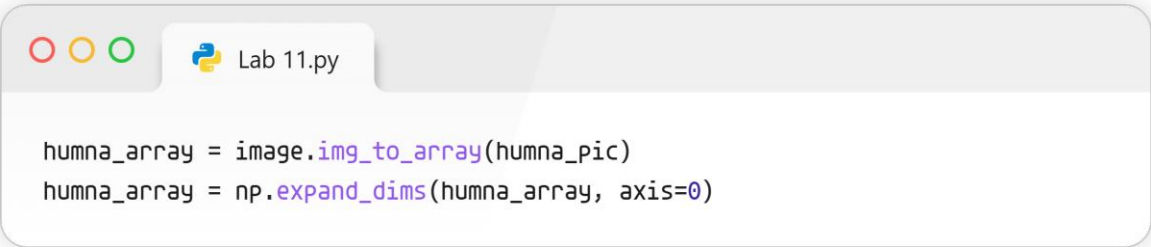
```
import tensorflow as tf
import numpy as np
from tensorflow.keras.preprocessing import image
```

A code editor window titled "Lab 11.py" with a Python icon. It contains the following code:

```
humna_mega_brain = tf.keras.applications.ResNet50V2(weights='imagenet')
```

A code editor window titled "Lab 11.py" with a Python icon. It contains the following code:

```
humna_pic = image.load_img('animal.jpg', target_size=(224, 224))
```

A code editor window titled "Lab 11.py" with a Python icon. It contains the following code:

```
humna_array = image.img_to_array(humna_pic)
humna_array = np.expand_dims(humna_array, axis=0)
```



Lab 11.py

```
humna_ready_pic = tf.keras.applications.resnet_v2.preprocess_input(humna_array)
```



Lab 11.py

```
humna_guesses = humna_mega_brain.predict(humna_ready_pic)
humna_results = tf.keras.applications.resnet_v2.decode_predictions(humna_guesses, top=3)

print("The mega brain's top 3 guesses are:", humna_results)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50v2\_weights\_tf\_dim\_ordering\_tf\_kernels\_102869336/102869336 1s 0us/step
1/1 2s 2s/step
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet\_class\_index.json
35363/35363 0s 0us/step
The mega brain's top 3 guesses are: [('n02422699', 'impala', np.float32(0.27407184)), ('n02391049', 'zebra', np.float32(0.21110192)),
```



Lab 11.py

```
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions
import numpy as np
import matplotlib.pyplot as plt
import requests
from PIL import Image
import io
from urllib.parse import urlparse
import urllib.request

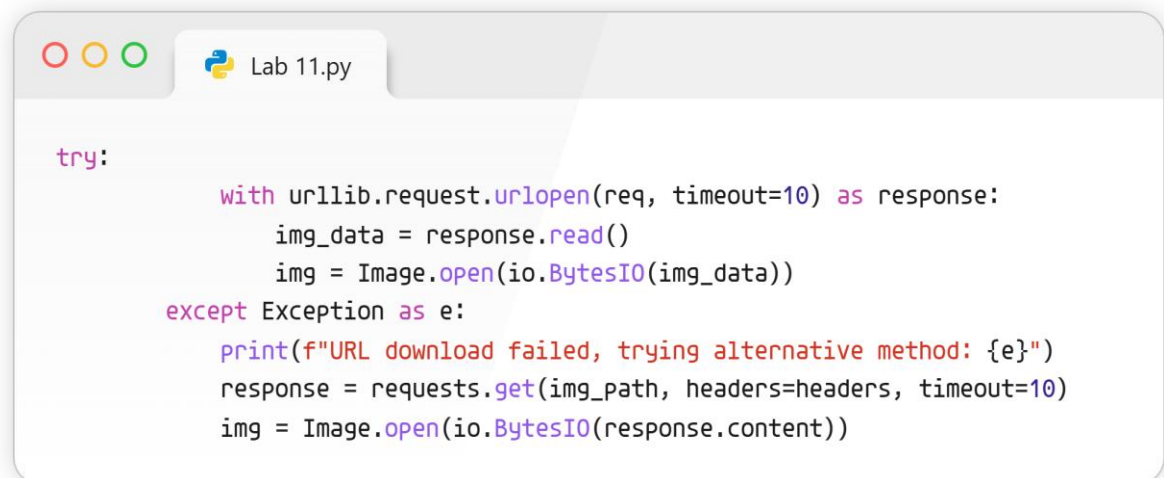
print("Loading MobileNetV2 with ImageNet weights...")
model = MobileNetV2(weights='imagenet', input_shape=(224, 224, 3))
print("Model loaded successfully!")

def load_and_preprocess_image(img_path, target_size=(224, 224)):
    """
```



```
Improved function to load image from various sources with better error handling
"""

try:
    # Check if it's a URL
    if img_path.startswith(('http://', 'https://')):
        # Create a request with a User-Agent header to avoid blocking
        headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'}
        req = urllib.request.Request(img_path, headers=headers)
```



```
try:
    with urllib.request.urlopen(req, timeout=10) as response:
        img_data = response.read()
        img = Image.open(io.BytesIO(img_data))
except Exception as e:
    print(f"URL download failed, trying alternative method: {e}")
    response = requests.get(img_path, headers=headers, timeout=10)
    img = Image.open(io.BytesIO(response.content))
```



```
else:
    img = Image.open(img_path)
    if img.mode != 'RGB':
        img = img.convert('RGB')
    img = img.resize(target_size, Image.Resampling.LANCZOS)
    x = np.array(img)
```

```
Lab 11.py

x = np.array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)
return x, img
except Exception as e:
    print(f"Error loading image: {e}")
```

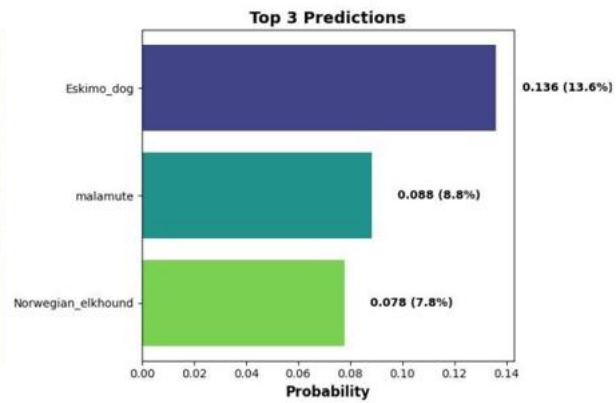
```
Lab 11.py

raise
def classify_image(img_path, model, top_k=3):
    """
    Classify an image using the pre-trained model and display top k predictions
    """
    try:
        x, original_img = load_and_preprocess_image(img_path)
        predictions = model.predict(x, verbose=0)
        decoded_predictions = decode_predictions(predictions, top=top_k)[0]
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
        ax1.imshow(original_img)
        ax1.set_title("Uploaded Image", fontsize=14, fontweight='bold')
        ax1.axis('off')
```

```
Lab 11.py

labels = [pred[1] for pred in decoded_predictions]
probabilities = [pred[2] for pred in decoded_predictions]
colors = plt.cm.viridis(np.linspace(0.2, 0.8, top_k))

y_pos = np.arange(len(labels))
bars = ax2.barh(y_pos, probabilities, color=colors)
ax2.set_yticks(y_pos)
ax2.set_yticklabels(labels, fontsize=10)
ax2.set_xlabel('Probability', fontsize=12, fontweight='bold')
ax2.set_title(f'Top {top_k} Predictions', fontsize=14, fontweight='bold')
ax2.invert_yaxis()
```



Summary

Instead of teaching a baby AI what a dog looks like, we borrowed an adult AI! We simply resized our image to fit the AI's "eyes," handed it the numbers, and the AI told us exactly what object was in the photo.

LAB 12: Simple Linear Regression with Scikit-Learn

What We Are Doing and Why

Computer Vision uses lots of Machine Learning. A very basic Machine Learning tool is **Linear Regression**. Imagine you plotted dots on a graph showing how many hours you studied and what score you got on a test. Linear Regression draws the best possible straight line through those dots so you can *predict* future scores!

The Code

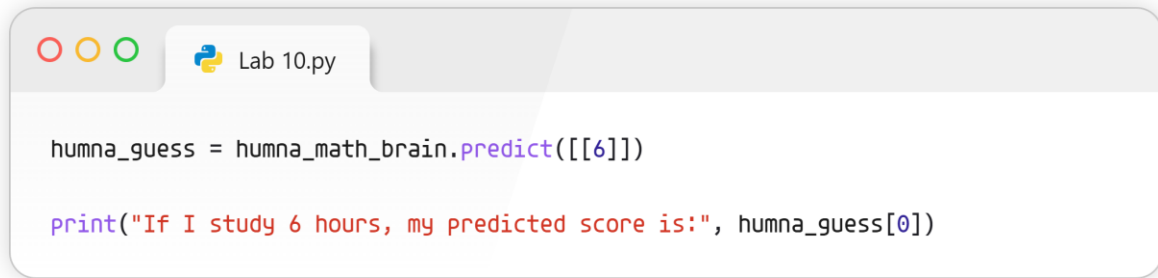
```
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression

humna_hours = np.array([[1], [2], [3], [4], [5]])
humna_scores = np.array([10, 20, 30, 40, 50])
```

```
humna_math_brain = LinearRegression()
```

```
humna_math_brain.fit(humna_hours, humna_scores)
```



```
humna_guess = humna_math_brain.predict([[6]])  
  
print("If I study 6 hours, my predicted score is:", humna_guess[0])
```

If I study 6 hours, my predicted score is: 59.99999999999999

Summary

We gave the computer a pattern: 1 hour = 10 points, 2 hours = 20 points, and so on. The computer drew a line to learn the rule. When we asked it what happens at 6 hours, it easily predicted 60 points! This same math helps AIs learn patterns in pictures.

Lab 13: Feature Extraction Using SIFT in OpenCV

What We Are Doing and Why

Have you ever looked at the night sky and found a constellation, like the Big Dipper? Even if you turn your head upside down or look from far away, you can still recognize it. A tool called **SIFT (Scale-Invariant Feature Transform)** does exactly this for pictures! It finds unique "star" spots (keypoints) in an image that the computer can easily recognize again, no matter if the picture is zoomed in, zoomed out, or spun around.

The Code

A code editor window titled 'Lab 14.py' with a Python icon. It contains two lines of Python code:

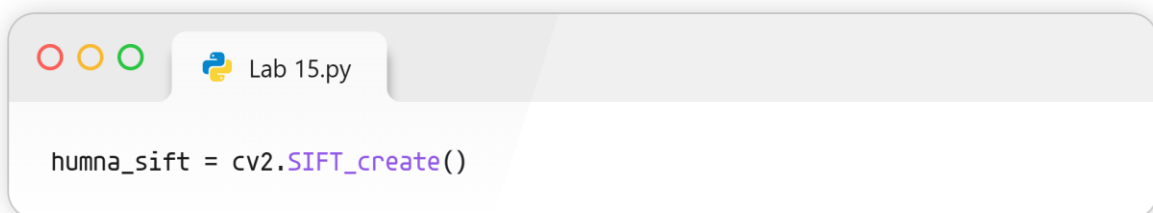
```
import cv2
import matplotlib.pyplot as plt
```

```
import cv2
import matplotlib.pyplot as plt
```

A code editor window titled 'Lab 14.py' with a Python icon. It contains two lines of Python code:

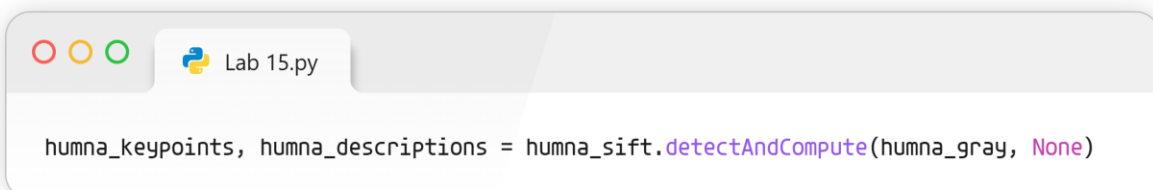
```
humna_image = cv2.imread('mountain.jpg')
humna_gray = cv2.cvtColor(humna_image, cv2.COLOR_BGR2GRAY)
```

```
humna_image = cv2.imread('mountain.jpg')
humna_gray = cv2.cvtColor(humna_image, cv2.COLOR_BGR2GRAY)
```

A code editor window titled 'Lab 15.py' with a Python icon. It contains one line of Python code:

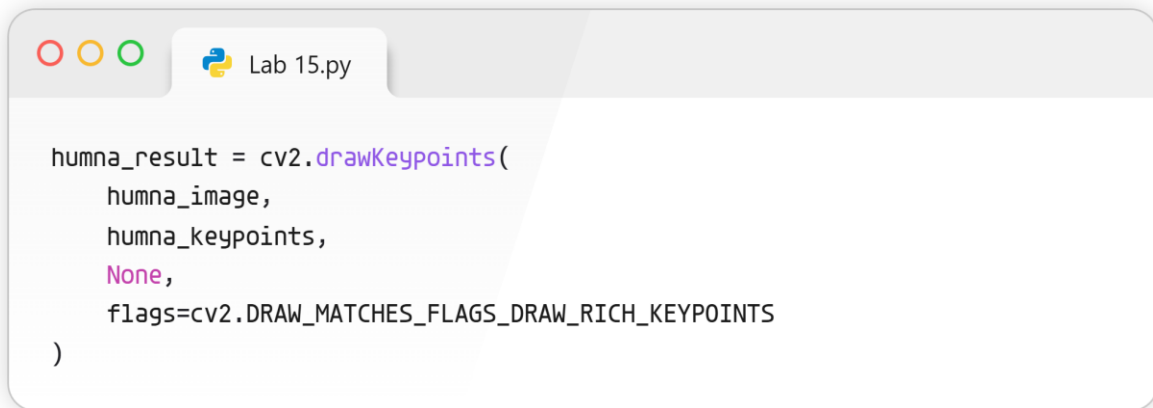
```
humna_sift = cv2.SIFT_create()
```

```
humna_sift = cv2.SIFT_create()
```

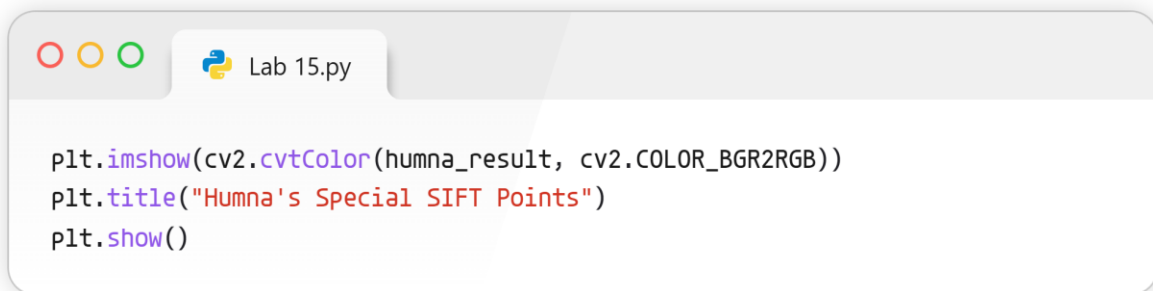
A code editor window titled 'Lab 15.py' with a Python icon. It contains one line of Python code:

```
humna_keypoints, humna_descriptions = humna_sift.detectAndCompute(humna_gray, None)
```

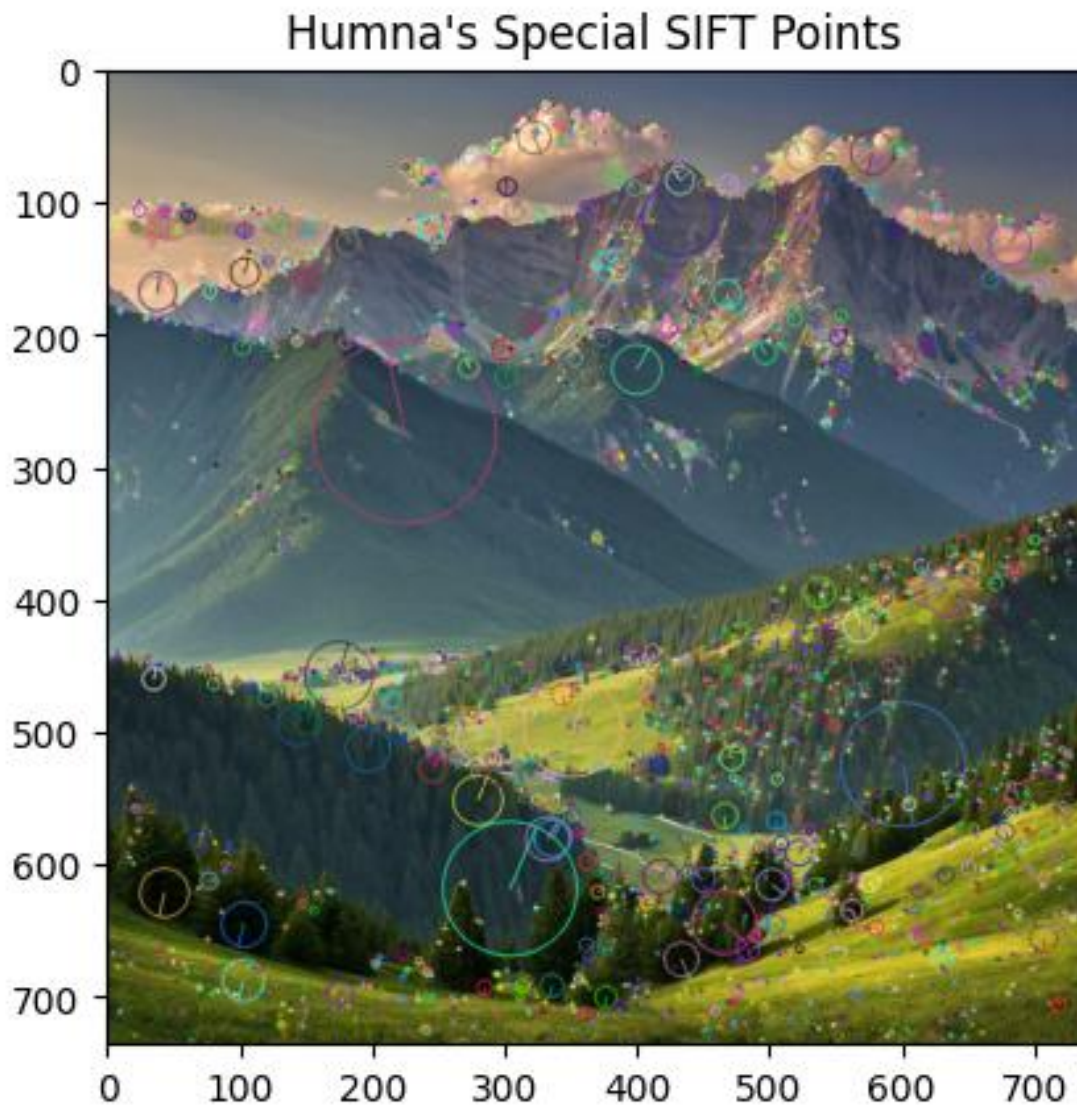
```
humna_keypoints, humna_descriptions = humna_sift.detectAndCompute(humna_gray, None)
```



```
humna_result = cv2.drawKeypoints(  
    humna_image,  
    humna_keypoints,  
    None,  
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS  
)
```



```
plt.imshow(cv2.cvtColor(humna_result, cv2.COLOR_BGR2RGB))  
plt.title("Humna's Special SIFT Points")  
plt.show()
```



Summary

SIFT acts like an extremely smart detective. Instead of looking at the whole image, it scans for very unique, interesting corners and bumps. It draws circles around these spots to remember them. Because it remembers these specific "constellations," it can find them again even if the image is rotated or resized!

What We Are Doing and Why

Sometimes colors trick the computer, so we need to look at textures instead. **HOG** (Histogram of Oriented Gradients) is a tool that looks at the *direction* of edges. It asks, "Are the lines pointing up, down, or sideways?" This is extremely helpful for recognizing things with a specific shape, like a human standing up!

The Code



Lab 07.py

```
import cv2
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure
```



Lab 07.py

```
humna_image = cv2.imread('person.jpg')
humna_gray = cv2.cvtColor(humna_image, cv2.COLOR_BGR2GRAY)
```



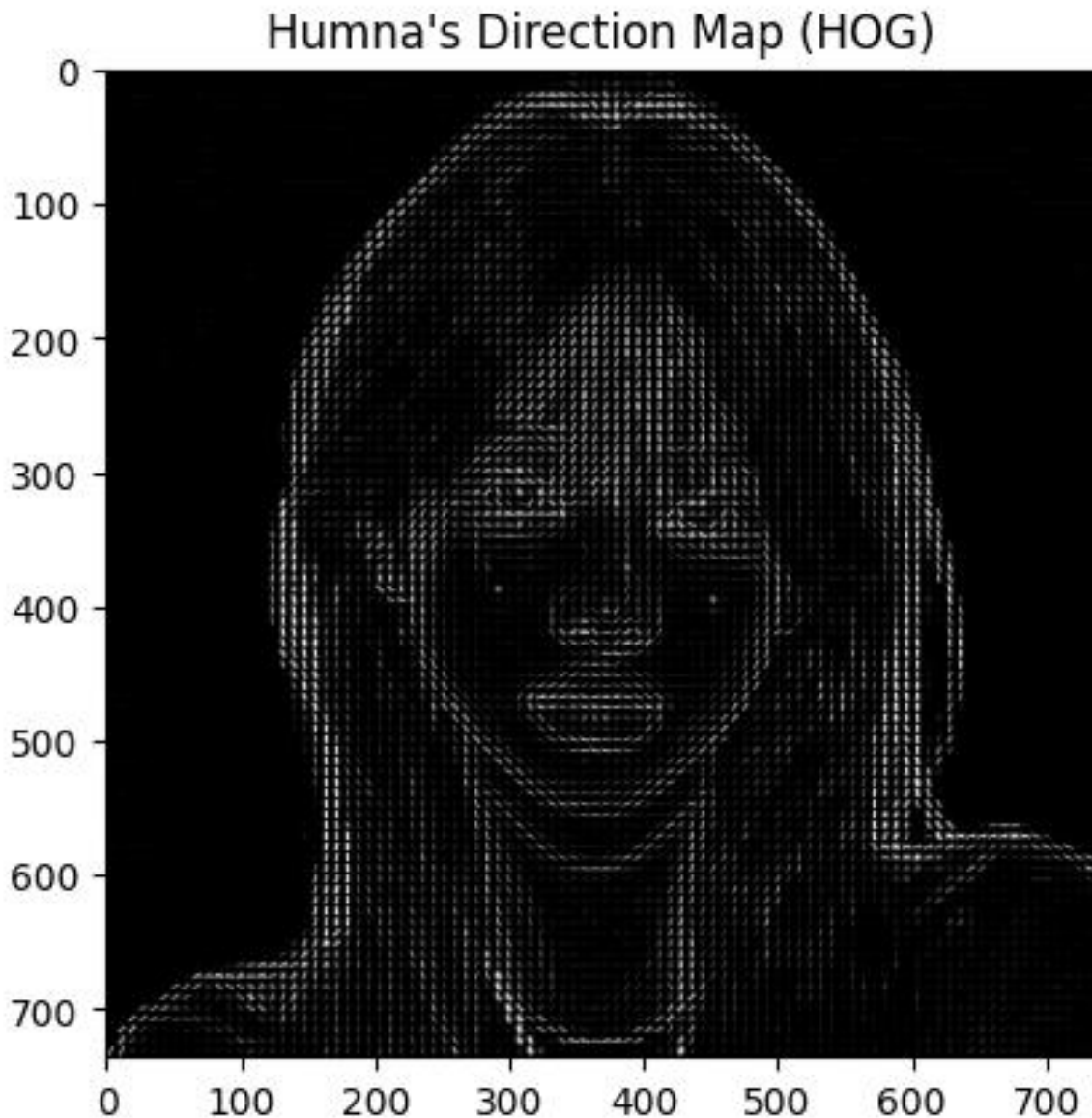
Lab 07.py

```
humna_features, humna_hog_pic = hog(humna_gray, visualize=True)
```



Lab 07.py

```
humna_bright_hog = exposure.rescale_intensity(humna_hog_pic, in_range=(0, 10))
plt.imshow(humna_bright_hog, cmap='gray')
plt.title("Humna's Direction Map (HOG)")
plt.show()
```



Summary

Instead of looking at the whole picture, HOG broke the picture down into tiny blocks and drew little arrows showing which way the light and dark parts were pointing. It is like turning a photograph into a sketch made entirely of tiny arrows!

What We Are Doing and Why

Sometimes segmentation breaks an image into *too many* tiny puzzle pieces. We can use a **Region Adjacency Graph (RAG)** to fix this. RAG looks at pieces that are touching each other. If two touching pieces have almost the exact same color, RAG glues them together into one bigger piece!

The Code



Lab 08.py

```
from skimage import io, segmentation, color
from skimage.graph import rag_mean_color, cut_threshold
import matplotlib.pyplot as plt
```



Lab 08.py

```
humna_img = io.imread('puzzle.jpg')
```



Lab 09.py

```
humna_tiny_pieces = segmentation.slic(humna_img, n_segments=250)
```



Lab 09.py

```
humna_graph_map = rag_mean_color(humna_img, humna_tiny_pieces)
```



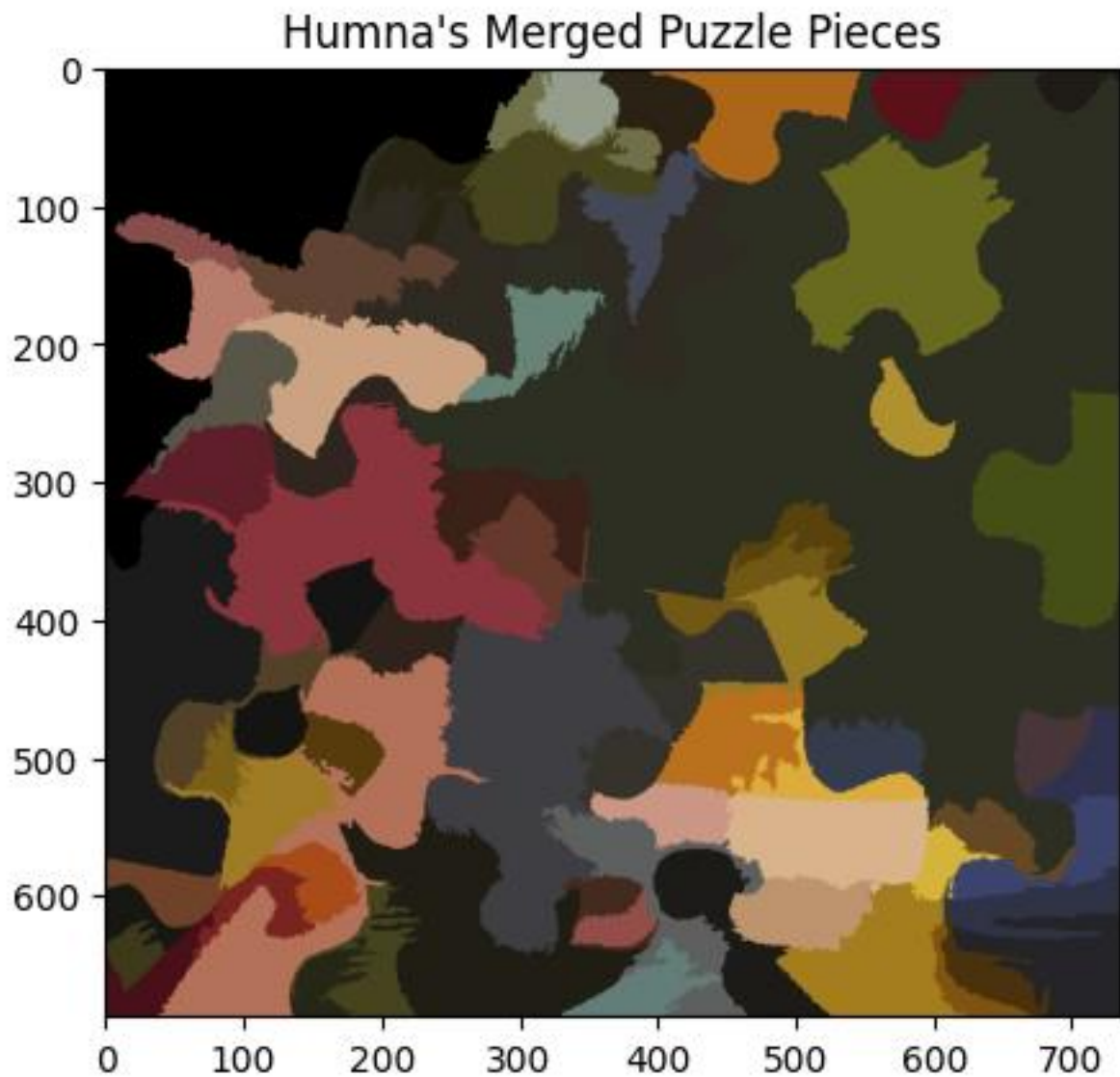
Lab 09.py

```
humna_merged_pieces = cut_threshold(humna_tiny_pieces, humna_graph_map, 29)
```



Lab 09.py

```
humna_final_image = color.label2rgb(humna_merged_pieces, humna_img, kind='avg')
plt.imshow(humna_final_image)
plt.title("Humna's Merged Puzzle Pieces")
plt.show()
```



Summary

First, we broke the image into 250 tiny chunks. Then, the computer built a map of the chunks and checked their colors. By gluing together neighbors with similar colors, we simplified the picture so the computer can easily spot the main objects.

LAB 15: Super-Fast Object Detection (YOLO)

What We Are Doing and Why

YOLO stands for "You Only Look Once." Unlike older AIs that had to scan an image over and over again to find things, YOLO looks at the picture one single time and finds all the objects instantly. It is so fast it can even be used on live drone cameras!

The Code



```
from ultralytics import YOLO
humna_yolo = YOLO('yolov8n.pt')
```



```
humna_results = humna_yolo('street.jpg')
humna_results[0].show()
```

```
Creating new Ultralytics Settings v0.0.6 file ✓
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% — 6.2MB 50.9MB/s 0.1s

image 1/1 /content/street.jpg: 640x448 7 persons, 9 cars, 533.2ms
Speed: 7.7ms preprocess, 533.2ms inference, 57.1ms postprocess per image at shape (1, 3, 640, 448)
```


Summary

This is the power of modern Computer Vision! With just three lines of code, we loaded a world-class AI, fed it an image, and it automatically drew perfect bounding boxes around people, cars, traffic lights, and more.